

UNIVERSIDADE DE SÃO PAULO
INSTITUTO DE MATEMÁTICA, ESTATÍSTICA E CIÊNCIA DA COMPUTAÇÃO
BACHARELADO EM MATEMÁTICA APLICADA E COMPUTACIONAL

**Análise formal de redes neurais para
detecção de intrusão**
Uma abordagem via lógica de Łukasiewicz

Matheus Oliveira da Silva

MONOGRAFIA FINAL
MAP2429 — TRABALHO DE
FORMATURA EM MATEMÁTICA APLICADA

Supervisor: Prof. Dr. Sandro Márcio da Silva Preto

São Paulo
2026

*O conteúdo deste trabalho é publicado sob a licença CC BY 4.0
(Creative Commons Attribution 4.0 International License)*

Em memória ao meu irmão, Misael de Oliveira Silva.

Agradecimentos

Primeiro, agradeço a Deus.

“O Senhor o deu, e o Senhor o tomou. Abençoado seja o nome do Senhor.”

(Jó 1:21)

Agradeço aos professores que fizeram a diferença durante a minha graduação. Em especial, ao Prof. Dr. Sandro Preto, que aceitou me orientar neste trabalho.

Agradeço à minha família: minha mãe, Márcia, meu pai, Aguinaldo, meus irmãos, Moisés e Misael, e minha namorada, Thainá, que são a minha base.

Agradeço, também, aos meus amigos de curso: Ana, Gustavo, Júlio, Lucas e Manuella, que me mostraram como a matemática pode ser bela e assumir diversas formas.

Resumo

Matheus Oliveira da Silva. **Análise formal de redes neurais para detecção de intrusão: Uma abordagem via lógica de Łukasiewicz**. Monografia (Bacharelado). Instituto de Matemática, Estatística e Ciência da Computação, Universidade de São Paulo, São Paulo, 2026.

O principal objetivo deste trabalho é o estudo e a aplicação da análise formal de redes neurais, com ênfase em perceptrons de multcamadas (MLP) que performam a detecção de intrusão em redes digitais. A fundamentação metodológica baseia-se na tradução dessas redes em funções racionais de McNaughton e sua representação na lógica de Łukasiewicz infinitamente-valorada, conforme a abordagem proposta por [PRETO \(2022\)](#).

Para a verificação, foram treinados modelos simplificados e formuladas propriedades de segurança críticas para o domínio. Os resultados da verificação formal demonstraram, para algumas interpretações, que as redes neurais analisadas falharam em garantir propriedades lógicas relevantes para a segurança do sistema. Conclui-se, portanto, que a análise formal se apresenta como uma ferramenta indispensável para a explicabilidade e confiabilidade de modelos de inteligência artificial em contextos críticos, revelando falhas que métricas de desempenho tradicionais não são capazes de detectar.

Palavras-chave: Redes Neurais. Perceptron de Multcamadas. Análise Formal. Lógica de Łukasiewicz. Funções de McNaughton. Explicabilidade. Alcançabilidade.

Abstract

Matheus Oliveira da Silva. **Formal analysis of neural networks for intrusion detection: *An approach via Łukasiewicz logic***. Capstone Project Report (Bachelor). Institute of Mathematics, Statistics and Computer Science, University of São Paulo, São Paulo, 2026.

The main objective of this work is the study and application of the formal analysis of neural networks, specifically multilayer perceptrons (MLP) performing intrusion detection tasks in digital networks. The methodological foundation is based on translating these networks into rational McNaughton functions and their representation in infinite-valued Łukasiewicz logic, following the approach proposed by [PRETO \(2022\)](#).

For verification purposes, simplified models were trained and critical security properties were formulated for the domain. The formal verification results demonstrated, for some interpretations, that the analyzed neural networks failed to guarantee logical properties relevant to system security. Therefore, it is concluded that formal analysis stands as an indispensable tool for the explainability and reliability of artificial intelligence models in critical contexts, revealing failures that traditional performance metrics are unable to detect.

Keywords: Neural Networks. Multilayer Perceptron. Formal Analysis. Łukasiewicz Logic. McNaughton Functions. Explainability. Reachability.

Lista de abreviaturas

IoT	Internet das Coisas (<i>Internet of Things</i>)
TCP	Protocolo de Controle de Transmissão (<i>Transmission Control Protocol</i>)
IP	Protocolo de Internet (<i>Internet Protocol</i>)
UDP	Protocolo de Datagrama de Usuário (<i>User Datagram Protocol</i>)
IDS	Sistema de Detecção de Intrusão (<i>Intrusion Detection System</i>)
HTTP	Protocolo de Transferência de Hipertexto (<i>Hypertext Transfer Protocol</i>)
HTTPS	Protocolo de Transferência de Hipertexto Seguro (<i>Hypertext Transfer Protocol Secure</i>)
FTP	Protocolo de Transferência de Arquivos (<i>File Transfer Protocol</i>)
SSH	Shell Seguro (<i>Secure Shell</i>)
CIC	Instituto Canadense de Cibersegurança (<i>Canadian Institute for Cybersecurity</i>)
DoS	Negação de Serviço (<i>Denial of Service</i>)
DDoS	Negação de Serviço Distribuída (<i>Distributed Denial of Service</i>)
ANN	Redes Neurais Artificiais (<i>Artificial Neural Networks</i>)
MLP	Perceptron de Multicamadas (<i>Multilayer Perceptron</i>)
CPL	Lógica Proposicional Clássica (<i>Classic Propositional Logic</i>)
MODSAT	Módulo Satisfatibilidade
ReLU	Unidade Linear Retificada (<i>Rectified Linear Unity</i>)
MILP	Programação Linear Inteira Mista (<i>Mixed-Integer Linear Programming</i>)

Lista de símbolos

$\mathbb{L}_\infty$	Lógica de Łukasiewicz Infinitamente-Valorada
$\mathbb{L}_\infty\text{-MODSAT}$	Lógica de Łukasiewicz Infinitamente-Valorada de Semântica Módulo Satisfatibilidade

Lista de figuras

1.1	Modelo de camadas da rede (TCP/IP)	5
2.1	Modelo neurônio	12
2.2	MLP do Exemplo 2.2	16
2.3	Função de ativação ReLU	16
4.1	Estrutura dos MLPs treinados	31

Lista de tabelas

4.1	Número de registros de tráfego de rede por rótulo (<i>CIC-IDS2017</i> tratado)	30
4.2	Matriz de confusão	32
4.3	Relatório de classificação para o <i>MLP-0</i>	33
4.4	Relatório de classificação para o <i>MLP-1</i>	34
5.1	Comparação entre abordagens de representação <i>MODSAT</i>	35
5.2	Especificações do ambiente computacional utilizado nos experimentos.	36
5.3	Resultados dos testes de verificação formal para o <i>MLP-0</i>	41
5.4	Resultados dos testes de verificação formal para <i>MLP-1</i>	42

Lista de programas

2.1	Descida do gradiente	13
2.2	Retropropagação	14

Sumário

Introdução	1
1 Sistemas de detecção de intrusão	3
1.1 Cibersegurança	3
1.2 Redes de computadores	4
1.2.1 Arquitetura TCP/IP	4
1.2.2 Dados de redes de computadores	6
1.3 Sistemas de detecção de intrusão	6
1.3.1 Técnicas de detecção	7
1.3.2 Tipos de tecnologia	7
1.4 CIC-IDS2017	8
1.4.1 Infraestrutura das redes	8
1.4.2 Tráfego benigno	8
1.4.3 Perfis de ataque	9
2 Perceptrons de multicamadas	11
2.1 Redes neurais artificiais	11
2.1.1 Neurônios	11
2.1.2 Aprendizado de máquina	12
2.1.3 Perceptrons de multicamadas (MLP)	14
2.2 MLP para detecção de intrusão	17
2.3 Funções de McNaughton	17
3 Lógica de Łukasiewicz infinitamente-valorada	19
3.1 Lógicas proposicionais	20
3.1.1 Linguagem da lógica proposicional	20
3.1.2 Semântica da lógica proposicional	20
3.2 Lógica proposicional clássica	21
3.3 Lógica de Łukasiewicz infinitamente-valorada	21

3.4	Semântica módulo satisfatibilidade	22
3.5	Representação módulo satisfatibilidade	23
3.6	Algoritmos de representação MODSAT	24
3.6.1	Formato regional	24
3.6.2	MLPs no formato regional	25
3.6.3	MLP em MODSAT	25
3.7	Resolução de satisfatibilidade e consequência lógica	26
4	Treinamento das redes neurais	29
4.1	Preparação do <i>CIC-IDS2017</i>	29
4.2	Estrutura dos MLPs	30
4.3	Treinamento dos MLPs	31
5	Análise formal das redes neurais	35
5.1	Representação <i>MODSAT</i>	35
5.2	Construção de propriedades	36
5.2.1	Propriedade 1: Alcançabilidade	36
5.2.2	Propriedade 2: Saída selecionada	36
5.2.3	Propriedade 3: Saída selecionada com limitação do complemento	37
5.2.4	Propriedade 4: Saída selecionada isoladamente	38
5.3	Testes das propriedades	39
5.3.1	<i>MLP-0</i>	39
5.3.2	<i>MLP-1</i>	40
6	Conclusão	43
	Referências	45

Introdução

Com o advento da Era da *Internet* das coisas (IoT), a importância das *redes digitais* expande-se continuamente, beneficiando diversos setores da indústria, incluindo transporte, saúde, agricultura, energia e manufatura. A IoT está crescendo e evoluindo, otimizando a eficiência e produtividade de diversos setores globais, contendo uma vasta coleção de dados transferidos e armazenados, e incrementando a escala de complexidade das redes com a integração de novas tecnologias (SALAMA *et al.*, 2023).

Nesse contexto, há também o aumento do impacto e alcance de *ataques cibernéticos* nas redes de computadores, que podem comprometer os sistemas que dependem da infraestrutura digital. Indústrias e hospitais podem ter o seu funcionamento comprometido por uma exploração de vulnerabilidade. Também, dados sensíveis de indivíduos e companhias podem ser compartilhados, violando o direito à privacidade. Qualquer serviço dependente de redes digitais está suscetível a ataques cibernéticos e é necessário que os meios de defesa sejam confiáveis e consistentes.

Diante dessa motivação, no capítulo inicial, os *sistemas de detecção de intrusão* (IDS) são definidos. Adicionalmente, o conjunto de dados *CIC-IDS2017* é proposto para o treinamento de duas redes neurais que atuam como IDSs.

Em seguida, no Capítulo 2, os principais conceitos e técnicas de treinamento de redes neurais são expostos. Apresenta-se também o Teorema 2.1, resultado que justifica a escolha do *perceptron de multicamadas* (MLP) como estrutura para o treinamento das redes neurais por sua correspondência com *funções de McNaughton*.

No Capítulo 3, as *lógicas proposicionais*, em especial a lógica de Łukasiewicz infinitamente-valorada (L_∞), são definidas junto com a *representação módulo satisfatibilidade* que, em suma, permite a representação de *funções racionais de McNaughton* em L_∞ . Tal representação permite o teste de propriedades de redes neurais no formato de problemas de satisfatibilidade e consequência lógica.

Para a parte prática do trabalho, no Capítulo 4, são relatados os detalhes sobre o treinamento de duas redes neurais, sem o exercício de técnicas avançadas de aprendizado de máquina, com o intuito de gerar redes neurais simples para submetê-las à análise formal. E, também, o capítulo detalha os relatórios de performance das redes neurais que são baseados em métricas comuns no mundo das redes neurais.

Em seguida, no Capítulo 5, são apresentados os detalhes das representações das redes neurais em L_∞ , assim como as definições, testes e resultados de quatro propriedades aplicadas às redes neurais:

- Propriedade 1: verifica se as redes neurais atingem, no mínimo, um limiar constante;
- Propriedade 2: verifica quais classes as redes neurais aprenderam a classificar;
- Propriedades 3 e 4: verificam, de maneiras diferentes, o nível de certeza da classificação das redes neurais.

Seguindo as boas práticas no uso de inteligência artificial, neste trabalho foi utilizada a ferramenta *Google Gemini* para a revisão gramatical, pesquisa de artigos científicos e desenvolvimento de códigos.¹

¹ Códigos disponíveis publicamente no repositório do *GitHub*: <https://github.com/matheus-osilva/Formal-Analysis-of-IDS>

Capítulo 1

Sistemas de detecção de intrusão

Este capítulo visa apresentar a teoria de sistemas de detecção de intrusão e uma base de dados para a implementação de sistemas de detecção de intrusão utilizando o aprendizado de máquina. Para isso, os conceitos básicos de cibersegurança e seus principais termos são definidos na Seção 1.1. Em seguida, as redes de computadores são caracterizadas, na Seção 1.2, para que o leitor entenda a dinâmica do tráfego de rede e quais são os seus dados. Os sistemas de detecção de intrusão, suas técnicas e tecnologias são abordadas na Seção 1.3. Por fim, as características da base de dados de tráfego de redes *CIC-IDS2017*, produzida pela Universidade de New Brunswick, são descritas na Seção 1.4, justificando a sua escolha para o treinamento de duas redes neurais para detecção de intrusão.

1.1 Cibersegurança

A cibersegurança é fundamentada em três serviços centrais: *confidencialidade*, *integridade* e *disponibilidade* (M. BISHOP, 2005).

A confidencialidade é a privacidade de informações e recursos computacionais que nasce da necessidade de manter dados sensíveis ocultos ou proteger direitos autorais. O controle de acesso é feito para incrementar a confidencialidade de um sistema e pode incluir autenticação, autorização e até mesmo criptografia — que mantém a informação protegida exposta, mas não inteligível.

A integridade é o nível de confiança de dados e recursos, isto é, a característica de manterem-se intactos a mudanças indesejadas que podem, por exemplo, comprometer sua veracidade. Os principais meios de manutenção da integridade são o preventivo, que visa impedir que danos sejam causados aos dados ou recursos, e o de detecção para alertar quando a integridade foi comprometida total ou parcialmente.

Por fim, a disponibilidade é a capacidade de utilizar os dados ou recursos providos por um sistema. A indisponibilidade de um sistema compromete os serviços e recursos para qual o sistema foi desenvolvido. Mecanismos de defesa contra a indisponibilidade são desenvolvidos para monitorar ou tomar ações no ambiente do sistema quando ameaças são detectadas.

Ataques cibernéticos são ações, bem-sucedidas ou não, que visam violar a segurança de um sistema e, portanto, impactar um ou mais dos três serviços centrais da cibersegurança.

De acordo com KAUR *et al.* (2023), o crescimento acelerado da complexidade e da interconectividade dos sistemas digitais tem elevado a frequência e a severidade dos ataques cibernéticos. Dessa forma, torna-se evidente a relevância do estudo de métodos e estratégias contra ameaças digitais, incluindo a análise formal das ferramentas de proteção que assegura a confiabilidade necessária em um domínio crítico como o da cibersegurança.

Conhecidos os termos e conceitos principais da cibersegurança, partimos para um dos principais ambientes digitais que são afetados por ataques e que devem ser protegidos, por sua grande relevância.

1.2 Redes de computadores

A *Internet* é, possivelmente, o maior sistema de engenharia já criado pela humanidade. Possui centenas de milhares de dispositivos interconectados e bilhões de usuários (KUROSE e ROSS, 2017). Por ser um campo dinâmico, de extenso crescimento e de importância global, as redes de computadores são os principais alvos de ataques cibernéticos. Por isso, é importante entender quais são os fatores que caracterizam o ambiente de redes digitais, quais desses fatores representam vulnerabilidades que um atacante pode explorar e como utilizar essa informação para defender redes de possíveis intrusões.

Uma rede de computadores é uma infraestrutura de conexões entre diversos tipos de dispositivos (*hosts*) tradicionais — computadores pessoais, estações de trabalho e servidores — e não tradicionais — celulares, eletrodomésticos, videogames, carros, entre outros. Os dispositivos são conectados através de *enlaces de comunicação* por onde transmitem *pacotes de informação*.

A relação entre servidor e cliente é definida quando um dispositivo, o cliente, deseja obter informações ou serviços de outro dispositivo, o servidor. Após a solicitação de informação do cliente, o servidor segmenta a informação em pacotes de dados com cabeçalhos de identificação que são enviados para o cliente, que reconstrói a informação original. A *taxa de transmissão* de pacotes é a velocidade em que os pacotes de informação são enviados através das ligações da rede e as *rotas* são os caminhos que os pacotes percorrem.

Um *protocolo* define o formato e a ordem das mensagens trocadas entre dois ou mais dispositivos em comunicação, bem como as ações a serem tomadas na transmissão e/ou recebimento de uma mensagem ou outro evento. Diferentes tipos de protocolos são utilizados para realizar de forma correta e eficiente a tarefa de comunicação entre dispositivos.

1.2.1 Arquitetura TCP/IP

A arquitetura TCP/IP estrutura os componentes de uma rede de computadores em camadas. Cada protocolo pertence a uma camada que, por sua vez, fornece um tipo de serviço para a camada acima. Ou seja, cada camada fornece um serviço performando ações dentro de si e utilizando os serviços da camada abaixo.



Figura 1.1: Modelo de camadas da rede (TCP/IP)

Camada de aplicação

A camada de aplicação possui diversos protocolos que realizam serviços como a requisição e transferência de documentos ou a transferência de mensagens de correspondência eletrônicas (*e-mails*). As redes de computadores são criadas por conta das aplicações e suas utilidades e, por isso, a camada de aplicação é a última no modelo TCP/IP.

Camada de transporte

A camada de transporte é utilizada para transmitir mensagens entre aplicações. Os dois principais protocolos dessa camada são:

1. TCP: garante a confiabilidade (entrega garantida e integridade) e ordem da informação transmitida ao seu destino, e controla o fluxo de dados sincronizando a velocidade de transmissão do receptor e do transmissor. Também, segmenta mensagens longas em segmentos menores para fazer o controle de congestionamento de mensagens.
2. UDP: de forma contrária ao TCP, o protocolo UDP está preocupado com a velocidade e continuidade do transporte de mensagens, pois não é necessário estabelecer conexão, numerar os pacotes ou gerenciar o fluxo. Neste caso, é mais importante que o transporte não seja interrompido do que perder alguns pacotes de informação.

Camada de rede

É responsável por mover pacotes, conhecidos como *datagramas*, de um dispositivo para outro. Os protocolos da camada de transporte em um servidor enviam um segmento da camada de transporte e um endereço de destino à camada de rede. Os protocolos da camada de rede, então, fornecem o serviço de entrega do segmento para a camada de transporte no cliente. A camada de rede também possui protocolos de roteamento que encontram os caminhos que os datagramas devem percorrer. Na camada de rede temos o famoso protocolo de *internet* (IP), que define os campos dos datagramas e como os dispositivos e roteadores devem agir nesses campos.

Camada de enlace

Para transportar pacotes e datagramas de um dispositivo para outro, a camada de rede depende dos serviços da camada de enlace. Dessa forma, em cada dispositivo, o datagrama

é enviado para a camada de enlace que entrega o datagrama para o destino, retornando-o para a camada de rede. Como os datagramas podem atravessar diferentes dispositivos para chegar ao seu destino, diferentes protocolos de enlace são utilizados. Os exemplos mais comuns de protocolos de enlace são o de *Ethernet* e *Wi-Fi*.

Camada física

Seu objetivo é transportar todos os *bits* de informação de um dispositivo ao outro. Trata-se das conexões físicas entre os dispositivos, como os cabos de fibra ótica, cabos coaxiais, cabos de cobre, entre outros. Para cada tipo de conexão física há um protocolo responsável pela transmissão de bits.

1.2.2 Dados de redes de computadores

Uma vez que a arquitetura TCP/IP foi definida, podemos entender quais são as informações que as redes nos fornecem e que podem indicar ataques cibernéticos.

Principalmente nas camadas de rede e de transporte, há um extenso volume de dados que caracterizam o tráfego de rede. As principais informações que são encontradas são:

- Identificação do fluxo: informações básicas que definem o fluxo de informação. Como a porta de destino que identifica qual serviço está em uso.
- Duração e comportamento: informações sobre o tempo das transmissões. Como o tempo total de transmissão, o tempo ativo de transmissão, e o tempo de inatividade.
- Volume dos pacotes: informações sobre o tamanho e contagem dos pacotes enviados e recebidos.
- Vazão dos pacotes: velocidade que os pacotes são transportados.
- Sinais de controle: comportamento das conexões, não a informação transportada em si, mas o acordo de conexão entre os dispositivos.

É precisamente sobre estas características volumétricas e comportamentais dos pacotes que os sistemas de detecção de intrusão baseados em fluxo operam, buscando padrões que desviem da normalidade descrita pelos protocolos.

1.3 Sistemas de detecção de intrusão

Os sistemas de detecção de intrusão (IDSs) funcionam a partir da coleta de informação do sistema monitorado. Portanto, o volume de dados é grande, dependendo do tamanho do ambiente digital protegido. Em uma rede de computadores, por exemplo, um IDS pode monitorar características do tráfego de informação entre nós da rede. Ao analisar o registro de informações, o IDS aplica um método de verificação para classificar se o sistema está sob ataque ou se houve uma tentativa de ataque.

Os métodos de detecção de intrusão são caracterizados, principalmente, pela detecção baseada em assinatura e a detecção baseada em anomalia. Sistemas podem ser desenvolvidos e treinados para funcionarem a partir de, exclusivamente, um dos métodos ou de

forma híbrida. A decisão de qual método utilizar depende fundamentalmente do contexto e objetivo do sistema de detecção de intrusão, não havendo um método melhor para todos os casos.

Para entender os sistemas de detecção de intrusão, é necessário compreender o que significa um sistema não estar sob ataque/intrusão. Sistemas que não estão sob ataques exibem três principais características (M. BISHOP, 2005):

1. As ações de processos e usuários geralmente se conformam a um padrão estatisticamente previsível. É improvável que um usuário que faz apenas processamento de texto ao usar o computador realize uma função de manutenção do sistema;
2. As ações de usuários e processos não incluem sequências de comandos para subverter a política de segurança do sistema. Em teoria, qualquer sequência desse tipo é excluída; na prática, apenas sequências conhecidas por subverter o sistema podem ser detectadas;
3. As ações dos processos se conformam a um conjunto de especificações que descrevem as ações que os processos podem (ou não podem) realizar.

1.3.1 Técnicas de detecção

Detecção baseada em assinatura

A detecção baseada em assinatura requer uma base de conhecimento dos padrões de dados que ataques e ameaças deixam no ambiente digital. O IDS incorpora essa base de conhecimento em um conjunto de regras e, quando as informações do monitoramento são processadas pelo IDS, as regras são aplicadas para determinar se o padrão se encaixa com as assinaturas de ataques conhecidos. Caso um ataque ou ameaça potencial sejam detectados, uma possível intrusão é reportada pelo sistema.

É o método mais simples e eficaz para detectar e classificar ataques conhecidos, porém, é ineficaz na detecção de ataques desconhecidos e variações de ataques conhecidos (LIAO *et al.*, 2013).

Detecção baseada em anomalia

A detecção baseada em anomalia parte da Característica 1 de sistemas que não estão sob ataque, ou seja, do princípio que o sistema possui um comportamento “normal”. O IDS mede o desvio entre os dados monitorados e os valores esperados. Então, caso o desvio seja estatisticamente relevante, gera-se um alerta de potencial intrusão.

É o método mais eficaz para detectar ataques novos e desconhecidos, bem como abusos de privilégio. Em contrapartida, é ineficaz em classificar o tipo de ataque detectado e, dificilmente, os alertas são gerados no tempo correto (LIAO *et al.*, 2013).

1.3.2 Tipos de tecnologia

De acordo com LIAO *et al.* (2013), as classes de tecnologia para IDSs são baseadas onde são implementadas para inspecionar atividades suspeitas e quais tipos de eventos podem

reconhecer. As principais classes são:

1. *IDS baseado em Host* (HIDS): monitora e coleta as características de dispositivos que contém informações sensíveis, servidores funcionando em serviços públicos e atividades suspeitas;
2. *IDS baseada em redes digitais* (NIDS): através de sensores, coleta tráfego de rede em segmentos específicos, e analisa as atividades de aplicações e protocolos para reconhecer possíveis intrusões;
3. *IDS baseada em conexão sem fio* (WIDS): similar ao NIDS, mas coleta dados de redes sem fio;
4. *IDS baseada em comportamento de rede* (NBA): coleta tráfego de rede para reconhecer ataques através de fluxos de dados inesperados;
5. *IDS híbrida* (MIDS): adota mais de uma classe de tecnologia.

Uma vez que o papel dos IDSs foi abordado, vamos apresentar o conjunto de dados *CIC-IDS2017* criado com o intuito de ser um conjunto de dados para o treinamento de redes neurais que atuam como sistemas de detecção de intrusão.

1.4 CIC-IDS2017

O conjunto de dados *CIC-IDS2017* foi produzido pela Universidade de New Brunswick no artigo de [I. SHARAFALDIN et al. \(2018\)](#). Contém 80 características de tráfego de redes de computadores, produzidos a partir do monitoramento das redes em um período de cinco dias durante o uso normal (seu tráfego benigno) e durante a reprodução de ataques cibernéticos comuns, como o ataque de força bruta (*brute force*) ou a negação de serviço (*DoS*).

1.4.1 Infraestrutura das redes

Para produzir o conjunto de dados, duas redes de computadores foram projetadas e implementadas: a *rede vítima* e a *rede atacante*. A primeira, possuindo os equipamentos básicos e necessários: roteador, *firewall*, *switch* e diferentes versões dos sistemas operacionais mais comuns (*Windows*, *Linux* e *Macintosh*). A *rede atacante* possui roteador, *switch* e computadores com os sistemas operacionais *Kali Linux* e *Windows 8.1*.

1.4.2 Tráfego benigno

Com o intuito de gerar dados realistas e não expor informações sensíveis, a técnica *B-Profile* ([Iman SHARAFALDIN et al., 2017](#)) foi empregada para construir o tráfego benigno. Primeiro, o comportamento abstrato de 25 usuários foi extraído baseado nos protocolos HTTP, HTTPS, FTP, SSH e protocolos de e-mail. Após a derivação dos perfis dos usuários, um agente foi utilizado para gerar os eventos benignos de forma realista.

É fundamental que o tráfego benigno seja realista para que os IDSs baseados em anomalias consigam aprender o comportamento normal de uma rede de computadores e efetivamente identificar o tráfego que se desvia de maneira estatisticamente relevante.

1.4.3 Perfis de ataque

Seis perfis de ataques, entre os mais comuns, foram simulados durante o monitoramento:

- Ataque de força bruta: utiliza o método de tentativa e erro para acessar páginas e informações ocultas;
- Ataque *heartbleed*: explora uma vulnerabilidade da biblioteca de criptografia *OpenSSL* para extrair o conteúdo da requisição de uma vítima;
- *Botnet*: utiliza uma rede de computadores infectada e controlada por um atacante para realizar tarefas maliciosas;
- Ataque DoS: possui o intuito de negar, temporariamente, o acesso a um recurso ou máquina em uma rede de computadores. Tipicamente, um número massivo de requisições supérfluas causa a negação do serviço;
- Ataque DDoS: semelhante ao ataque DoS, porém ocorre de maneira distribuída, ou seja, diversas máquinas são utilizadas para fazerem requisições ao sistema;
- Ataques *Web*: no contexto do *CIC-IDS2017*, as simulações incluíram: injeção SQL, onde uma consulta maliciosa é enviada a um sistema; *cross-site scripting* (XSS), onde o atacante injeta *scripts* maliciosos; e força bruta via protocolo HTTP;
- Ataque de infiltração: após a exploração de uma aplicação vulnerável, por exemplo, o atacante ganha acesso interno que facilita a condução de diferentes tipos de ataques na rede da vítima.

A escolha do conjunto de dados *CIC-IDS2017*, para treinar duas redes neurais utilizando o aprendizado de máquina supervisionado, justifica-se por sua cobertura dos ataques mais comuns, dados sobre o tráfego benigno e 80 características, que possibilita a criação de eficientes IDSs baseados em anomalia de rede (NBA). E, além disso, o *CIC-IDS2017* foi produzido para atender diversos critérios de avaliação: configuração completa de rede, coleta de tráfego completa, protocolos comuns disponíveis, diversidade de ataques e conjunto de características.

Após abordar o problema (ataques cibernéticos), o ambiente (redes de computadores), a ferramenta de proteção (sistemas de detecção de intrusão) e os materiais para sua construção (*CIC-IDS2017*), o próximo capítulo explora as técnicas de redes neurais necessárias para a implementação de IDSs básicos e eficientes.

Capítulo 2

Perceptrons de multicamadas

Este capítulo visa apresentar a teoria de perceptrons de multicamadas (MLPs) e a sua conexão direta com as funções de McNaughton. Para isso, a Seção 2.1 introduz a definição de redes neurais artificiais. Em seguida, para a compreensão do funcionamento e implementação de redes neurais artificiais, a Seção 2.1.1 dedica-se à definição de sua estrutura fundamental, o neurônio, e a Seção 2.1.2 é responsável por mostrar como as redes neurais aprendem a realizar uma tarefa. Logo após a compreensão de redes neurais artificiais de forma generalista, os perceptrons de multicamadas, escolhidos para criar dois sistemas de detecção de intrusão, são definidos junto com o efeito caixa-preta que justifica a necessidade de análise formal de redes neurais. Finalmente, para a eventual introdução da análise formal no próximo capítulo, a correspondência entre funções de McNaughton e redes neurais é demonstrada na Seção 2.3.

2.1 Redes neurais artificiais

Redes neurais artificiais (ANN) são modelos computacionais cuja inspiração conceitual remonta à estrutura e ao funcionamento dos cérebros biológicos. Possuem unidades básicas de processamento de informação, os *neurônios*, que se relacionam por *conexões*, comportando-se como grafos direcionados. O princípio fundamental das redes neurais reside na capacidade de suas conexões, ponderadas por pesos ajustáveis, permitirem que a rede seja submetida a um processo de *treinamento* para mapear relações complexas e, em geral, são utilizadas para identificar padrões sobre um conjunto de dados (*dataset*) e representar uma função f^* . Por exemplo, a partir de um conjunto de dados de tráfegos de uma rede de computadores, uma rede neural pode ser treinada para identificar tráfegos anômalos, gerando um alerta de possível ataque cibernético. Para alcançar tal capacidade, a rede é treinada com o objetivo de modelar a correlação entre as *características* (features) do conjunto de dados e a classificação esperada.

2.1.1 Neurônios

De acordo com C. BISHOP (2016), em uma rede neural, o neurônio recebe um sinal de entrada, um vetor de n dimensões, com $n \in \mathbb{N}^*$, $X^T = [x_1, x_2, \dots, x_n]$, que pode ser a saída

de outro neurônio ou uma informação do vetor de entrada. Cada valor de x é associado a um valor do vetor de pesos, de n dimensões, $W^T = [w_1, w_2, \dots, w_n]$ que representa a sensibilidade do neurônio a cada um dos sinais de entrada.

Seja o vetor de entrada $X \in \mathbb{R}^n$ e um vetor de pesos $W \in \mathbb{R}^n$, associados a um único neurônio. A *ativação* do neurônio, $a \in \mathbb{R}$, é calculada através de uma transformação afim, que inclui um termo de viés $b \in \mathbb{R}$:

$$a = b + W^T X = b + \sum_{i=1}^n w_i \cdot x_i. \quad (2.1)$$

Por fim, uma função de ativação h , diferenciável e não-linear, é aplicada sobre a para gerar a saída z do neurônio, que posteriormente será utilizada como entrada para outro neurônio ou como um valor do vetor de saída da rede neural:

$$z = h(a). \quad (2.2)$$

A não-linearidade é essencial para que a rede neural possa aproximar funções não-lineares, caso contrário a rede neural seria um modelo de regressão linear. A diferenciabilidade é importante para os algoritmos de aprendizado que utilizam o método de otimização *descida de gradiente* (BURKOV, 2019). Para os algoritmos de aprendizado, a função de ativação precisa ser diferenciável em todo o seu domínio ou na vasta maioria de pontos dele.

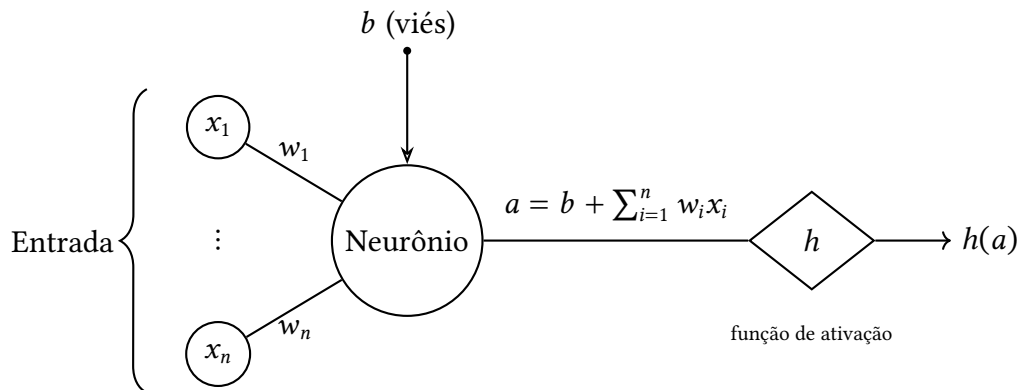


Figura 2.1: Modelo neurônio

2.1.2 Aprendizado de máquina

O *aprendizado de máquina* consiste no processo de ajuste dos parâmetros de uma rede neural para que ela execute uma tarefa específica, utilizando um conjunto de dados de treinamento. Técnicas de otimização são implementadas, como o algoritmo de descida do gradiente, para minimizar uma função de custo que quantifica a discrepância entre a saída da rede neural e a função-alvo f^* .

Aprendizado supervisionado

O aprendizado de máquina *supervisionado* é realizado a partir de um conjunto de dados de treinamento rotulado. Tal conjunto é formado por uma ou mais características (*features*) de um elemento e um rótulo (*label*) que o classifica ou representa algum valor numérico de interesse, é representado pelo par $\{(X_i, y_i)\}_{i=1}^N$, onde X_i é o vetor de características e y_i o seu rótulo. O objetivo desse tipo de aprendizado é modelar a relação entre as características e o rótulo dos elementos. O objetivo do treinamento é alcançado quando o modelo demonstra a capacidade de generalizar, rotulando corretamente elementos não vistos a partir de suas características (BURKOV, 2019).

Por exemplo, considerando um conjunto de dados de treinamento em que cada elemento é uma pessoa e o vetor de características possui informações descritivas — como a quantidade de açúcar ingerida ou quantidade de horas de exercícios físicos praticados no último ano, idade, etc — e o rótulo é um valor numérico: 1 se a pessoa já teve um ataque cardíaco e 0 caso contrário. Tal conjunto pode ser utilizado para treinar um modelo que, ao receber um vetor de características X , deduz a probabilidade de a pessoa ter um ataque cardíaco.

Aprendizado não supervisionado

O aprendizado de máquina *não supervisionado* utiliza um conjunto de dados de treinamento sem rótulo, ou seja, $\{X_i\}_{i=1}^N$. Da mesma forma, o vetor X_i carrega as características do elemento do conjunto. O objetivo é treinar um modelo que, a partir de um vetor X de entrada, transforma-o em outro vetor ou valor que pode ser utilizado para resolver um problema prático (BURKOV, 2019).

Por exemplo, pode ser utilizado para, a partir de características de uma transferência de dados em uma rede de computadores, identificar se o tráfego é anômalo ou se faz parte do conjunto de transferências típicas.

Algoritmos de aprendizado

O algoritmo de descida do gradiente é um método iterativo de otimização baseado em cálculo diferencial para minimizar uma função de custo J . A partir de um ponto inicial, o algoritmo calcula o gradiente negativo da função de custo no ponto atual. Este vetor indica a direção de máximo declínio local da função. Um passo em direção à minimização é tomado proporcionalmente à taxa de aprendizado η , definida no algoritmo, e novamente o gradiente negativo é calculado indicando a próxima direção para se aproximar do mínimo local.

Algoritmo 2.1 Descida do gradiente

- 1: Calcular o gradiente negativo $d_k = -\nabla J(W^k)$
 - 2: Tomar próximo passo $W^{k+1} = W^k + \eta d_k$
-

O Algoritmo 2.1 foi adaptado de FRIEDLANDER (1994) pois, originalmente, não considera um tamanho de passo fixo e calcula-o a partir de um subproblema de otimização.

A rede neural, em cada etapa, é ativada a partir de uma entrada do conjunto de dados de treinamento e, pelo valor de saída, os pesos e vieses são reajustados na direção que minimiza a diferença entre a saída atual e a saída esperada da rede neural.

O passo crucial no algoritmo de descida do gradiente é o cálculo de $\nabla J(W^k)$. Para uma rede neural com múltiplas camadas, onde a função de custo é uma composição complexa de funções, este cálculo seria computacionalmente proibitivo se feito de forma ingênua. O algoritmo de retropropagação (*backpropagation*) surge como uma solução eficiente para esse problema.

Algoritmo 2.2 Retropropagação

- 1: Aplicar um vetor de entrada na rede neural e, usando (2.1) e (2.2), propagar a informação para encontrar as ativações de todos os neurônios e de sua saída;
- 2: Calcular o erro na última camada comparando a saída gerada com o valor esperado do conjunto de treinamento;
- 3: Retropropagar o erro calculado, das camadas finais para as iniciais, para determinar a contribuição para o erro total de cada neurônio nas camadas ocultas;
- 4: Calcular os gradientes necessários para indicar a direção e magnitude de ajuste da rede neural a partir do erro total.

Algoritmo adaptado de: C. BISHOP, 2016

2.1.3 Perceptrons de multicamadas (MLP)

Perceptron

O *perceptron*, introduzido em ROSENBLATT (1958), é um neurônio que possui função de ativação degrau, isto é, segue a mesma ativação em (2.1), mas a sua saída é representada por:

$$f_{\phi}(a) = \begin{cases} 1 & \text{se } a \geq \phi \\ -1 & \text{se } a < \phi \end{cases}, \quad (2.3)$$

onde ϕ é o limiar constante.

O perceptron é um regressor linear e, portanto, não consegue representar diversas funções, como a função “ou-exclusivo” definida como:

$$f(x, y) = \begin{cases} 0 & \text{se } x = y \\ 1 & \text{se } x \neq y \end{cases}$$

Para superar tais limitações os perceptrons de multicamadas foram introduzidos.

Estrutura dos MLPs

Os *perceptrons de multicamadas* (MLPs), ou *multilayer perceptrons*, são as chamadas redes neurais de *feedforward*, assim denominadas por possuírem um fluxo de informação estritamente unidirecional, propagando-se da camada de entrada em direção à camada de saída. Para $n, m \in \mathbb{N}^*$ e $k \in \mathbb{N}$, o MLP é composto por uma camada de entrada de dimensão n , k camadas ocultas e uma camada de saída de dimensão m . A informação propaga-se sequencialmente do vetor de entrada X através de uma cadeia de composição

de funções $f^{(1)}, f^{(2)}, \dots, f^{(k)}$,¹ que representam as *camadas ocultas*, culminando no vetor de saída $Y = f^{(k+1)}$. Nesse caso, a função $f^{(1)}$ é chamada de primeira camada, a função $f^{(2)}$ de segunda camada e assim por diante.

O fluxo da informação pode ser representado pela expressão da saída Y :

$$Y = f^{(k+1)} \left(\dots f^{(2)} \left(f^{(1)}(X) \right) \dots \right). \quad (2.4)$$

Cada camada oculta l , com $l \in \{1, \dots, k\}$, possui $n_l \in \mathbb{N}^*$ neurônios. E, a partir, da definição de ativação de um neurônio (2.1) e sua saída (2.2), define-se a ativação de um neurônio j , com $j = 1, \dots, n_l$, da camada oculta l por

$$f_j^{(l)}(X, W) = z_j^{(l)} = h(a_j^{(l)}) = h\left(b_j^{(l)} + \sum_{i=1}^{n_{l-1}} w_{ji}^{(l)} x_i^{(l-1)}\right) = x_j^{(l)}, \quad (2.5)$$

sendo $b_j^{(l)}$ e $w_{ji}^{(l)}$ valores do vetor de parâmetros W onde $b_j^{(l)}$ é o viés do neurônio j da camada l e $w_{ji}^{(l)}$ é o peso do neurônio j da camada l referente à entrada de índice i .

Cada camada oculta, composta por n_l neurônios, possui cada neurônio totalmente conectado a todos os neurônios da camada anterior e à todos os neurônios da camada posterior. No caso da primeira camada, os neurônios são conectados ao vetor de entrada e à segunda camada; no caso da última camada, os neurônios são conectados à penúltima camada e ao vetor de saída.

Os neurônios das camadas ocultas atuam como sensores em série, recebem toda informação disponível pela camada anterior e geram um único valor que é utilizado pela próxima camada.

Exemplo 2.1. O caso mais fundamental de um perceptron de multicamadas possui apenas um vetor de entrada e um vetor saída, ambos com dimensão 1. Sua saída y é dada por

$$y = f^{(1)}(x) = h(b + wx).$$

Exemplo 2.2. Para visualizar a estrutura dos neurônios entre as camadas, considere o exemplo em que $n = 3, m = 3, k = 2$, e que $n_1 = n_2 = 5$. Nesse caso, a expressão (2.4) é

$$Y = f^{(3)} \left(f^{(2)} \left(f^{(1)}(X) \right) \right),$$

e a sua estrutura está ilustrada na Figura 2.2

¹ A notação $f^{(n)}$, neste trabalho, trata-se da função que representa a camada n . Não confundir com a n -ésima derivada da função f .

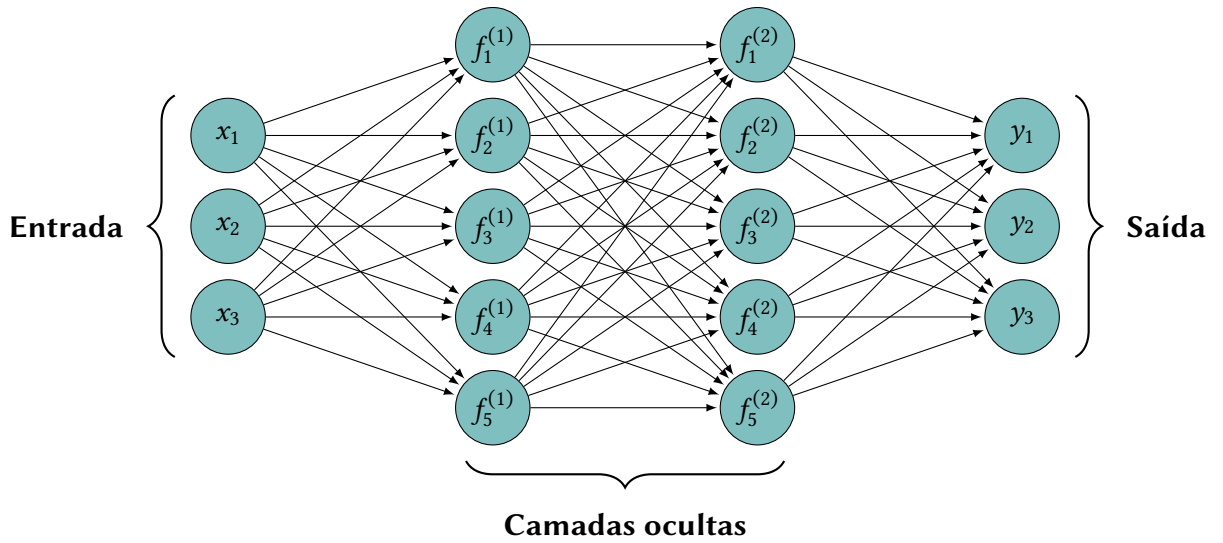


Figura 2.2: MLP do Exemplo 2.2

Não-linearidade e efeito caixa-preta

Para aproximar funções não-lineares é necessário escolher uma função de ativação que, aplicada à transformação linear feita pelo neurônio, gere uma saída não-linear. Segundo [GOODFELLOW *et al.* \(2016\)](#), a recomendação padrão é usar a unidade linear retificada (ReLU). A unidade linear retificada é a função $h(z) = \max(0, z)$ e, apesar de adicionar a não-linearidade, é uma função linear por partes e, por isso, garante certas propriedades que facilitam a otimização nos algoritmos de aprendizado que utilizam o gradiente e a sua representação em lógica de Łukasiewicz que será abordada adiante.

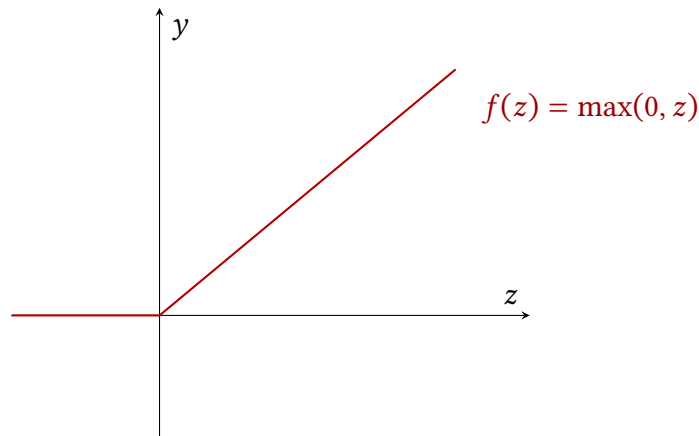


Figura 2.3: Função de ativação ReLU

Pelo fato de as camadas ocultas não acessarem diretamente a informação do conjunto de dados de treinamento e por sua não-linearidade, ocorre o efeito *caixa-preta* nas redes neurais e, em especial, em perceptrons de multicamadas. Tal efeito é a falta de *explicabilidade* da informação aprendida pela rede neural, isto é, não é possível, de forma direta, apreender a relação entre os parâmetros das camadas ocultas e as *características* do conjunto. A explicabilidade de redes neurais, principalmente em sistemas críticos, é um dos critérios

para a sua implementação pois essa opacidade pode minar a confiança de especialistas no sistema e, ademais, a dificuldade em prever o comportamento do modelo em cenários inéditos pode resultar em decisões indesejadas ou mesmo danosas (CAPUANO *et al.*, 2022).

Há diversos métodos para amenizar ou solucionar os problemas gerados pelo efeito caixa-preta. O método que será abordado neste trabalho é o de análise formal de redes neurais. Isto é, através de ferramentas matemáticas, uma representação exata da rede neural é construída em um *sistema lógico* e, a partir deste, propriedades sobre a rede neural podem ser codificadas e testadas.

2.2 MLP para detecção de intrusão

Os NIDS podem ser implementados utilizando ferramentas como o *Snort*, que captura o tráfego de rede e compara-o com assinaturas de ataques conhecidos. Todavia, a base de assinaturas deve ser atualizada constantemente para que a detecção continue eficaz apesar da criação de novos métodos e estratégias de ataques. Uma outra limitação é a de que a informação do ataque depende diretamente do ambiente utilizado (sistema operacional, computador, aplicações, etc).

Tais desvantagens podem ser contornadas ao utilizar-se o aprendizado de máquina para treinar um sistema de detecção baseado em anomalias (NBA). Tanto o aprendizado supervisionado quanto não supervisionado podem ser realizados neste caso pois há conjuntos de dados disponíveis para o treinamento, como o *CIC-IDS2017*.

As estruturas de redes neurais, como o perceptron de multicamadas, as redes neurais recorrentes ou as redes neurais convolucionais, podem ser utilizadas para empregar-se os IDSs. Todavia, como o objetivo do trabalho é empregar a análise formal para avaliar propriedades de uma rede neural, a escolha do MLP é dada por dois principais motivos: o MLP é adequado para a tarefa de realizar a detecção de intrusão em redes digitais, de acordo com ROSAY *et al.* (2020), onde a partir do *CIC-IDS2017* um MLP foi treinado obtendo-se uma acurácia de 99,41% com taxa de falso-positivo de 0,59%, e a relação direta entre MLPs e funções de McNaughton, que podem ser analisadas utilizando a lógica de Łukasiewicz (PRETO, 2022).

2.3 Funções de McNaughton

Uma *função de McNaughton*, descrita pelo **Teorema 2** de McNAUGHTON (1951), é uma função f que satisfaz as seguintes condições:

- O domínio de f é o hipercubo $[0, 1]^n$, a imagem de f possui dimensão 1 e está contida no intervalo $[0, 1]$, ou seja, $f : [0, 1]^n \rightarrow [0, 1]$, com f contínua;
- Há um número finito de polinômios $\lambda_1, \dots, \lambda_\mu$, onde $\lambda_j = b_j + m_{1j}x_1 + \dots + m_{nj}x_n$, onde todos os b 's e m 's são inteiros tais que para todo $[x_1, \dots, x_n] \in [0, 1]^n$ há um índice $j \in \mathbb{N}$, com $1 \leq j \leq \mu$, tal que $f(x_1, \dots, x_n) = \lambda_j(x_1, \dots, x_n)$. Os polinômios $\lambda_1, \dots, \lambda_\mu$ são as *partes lineares* de f .

Uma *função racional de McNaughton* é aquela que permite coeficientes racionais em suas partes lineares.

Considere um MLP com as seguintes características:

- Possui função de ativação linear por partes, como a ReLU;
- Seus valores de entrada estão no hipercubo $[0, 1]^n$;
- Seus valores de saída estão no hipercubo $[0, 1]^m$;
- Seus coeficientes são inteiros.

Teorema 2.1. Se o MLP atende a todas as características acima então cada componente do seu vetor de saída é equivalente a uma função de McNaughton.

Esboço da demonstração. Pela definição em (2.5), a expressão da ativação de um neurônio j da primeira camada oculta é

$$x_j^{(1)} = h \left(b_j^{(1)} + \sum_{i=1}^n w_{ji}^{(1)} x_i \right).$$

Note que $x_j^{(1)}$ é linear por partes pois aplica uma função linear por partes em uma função linear, a função afim. E a composição de funções contínuas é contínua, assim como a composição de funções lineares com coeficientes inteiros é linear com coeficientes inteiros. As ativações dos neurônios das camadas posteriores também serão lineares por partes pois vão compor os valores das camadas anteriores da mesma forma. Portanto, os valores do vetor de saída também serão lineares por partes

$$f_j^{(k+1)}(X, W) = x_j^{(k+1)} = y_j = h \left(b_j^{(k+1)} + \sum_{i=1}^{n_k} w_{ji}^{(k+1)} x_i^{(k)} \right)$$

e, então, cada $f_j^{(k+1)} : [0, 1]^n \rightarrow [0, 1]$, para $j \in \{1, \dots, m\}$ é uma função de McNaughton. $\square$

Todavia, é muito improvável que os coeficientes de uma rede neural MLP sejam inteiros. Portanto, é necessário expandir tal correspondência para as funções racionais de McNaughton. Para que haja correspondência entre um MLP e funções racionais de McNaughton, basta que, além das condições descritas acima, os coeficientes do MLP sejam racionais em vez de inteiros.

Apresentada a relação entre perceptrons multicamadas e funções racionais de McNaughton, o sistema lógico multivalorado de Łukasiewicz será abordado como sistema lógico capaz de representar tais funções e, portanto, será utilizado para fazer a análise formal de duas redes neurais MLPs.

Capítulo 3

Lógica de Łukasiewicz infinitamente-valorada

Para o estudo da análise formal de redes neurais, a ferramenta principal de análise de propriedades, a lógica de Łukasiewicz infinitamente-valorada (L_∞), é estudada neste capítulo. Após uma definição geral da Lógica, na Seção 3.1 as lógicas proposicionais são definidas de forma geral. Em seguida, na Seção 3.2 a lógica proposicional clássica é explorada com o intuito didático de exemplificar uma lógica proposicional popular. Na Seção 3.3, a lógica de Łukasiewicz infinitamente-valorada é formalizada. Em seguida, nas Seções 3.4 e 3.5 a semântica e a representação módulo satisfatibilidade são introduzidas, pois são necessárias para que a representação de funções de McNaughton em L_∞ englobe, também, as funções racionais de McNaughton, que possuem conexão direta com MLPs. Na Seção 3.6, os algoritmos que constroem representações de funções racionais de McNaughton em L_∞ -MODSAT são discutidos. Por fim, na Seção 3.7 a alcançabilidade e consequência lógica, principais ferramentas deste trabalho para a análise de propriedades, são formalizadas, e o Algoritmo LUKASOL é abordado como ferramenta para resolver problemas de verificação formal.

Lógica é a ciência que estuda princípios e métodos de inferência, tendo o objetivo principal de determinar em que condições certas coisas se seguem (são consequências), ou não, de outras (MORTARI, 2001).

Para o emprego correto de tal ciência do pensamento dedutivo, uma *linguagem lógica* foi desenvolvida, a *lógica simbólica*. O intuito era acabar com imprecisões e paradoxos gerados pela linguagem natural para obter-se uma linguagem mais rigorosa. Para tal, a linguagem deveria possuir regras claras e objetivas de formação de sentenças (*sintaxe*), de modo a limitar a linguagem para controlá-la (ENDERTON, 2001). Além disso, o significado de uma sentença da lógica simbólica depende exclusivamente de sua interpretação (*semântica*), como na linguagem proposicional onde a interpretação é dada por uma função valoração.

A seguir, descreveremos as lógicas proposicionais para fornecer a teoria base para abordar a lógica de Łukasiewicz infinitamente-valorada, uma lógica proposicional, que, diferentemente da lógica proposicional clássica, possui valores reais no intervalo $[0, 1]$, tornando-a ideal para modelar o comportamento de neurônios artificiais.

3.1 Lógicas proposicionais

As lógicas proposicionais possuem sintaxe e semântica independentes, diferentemente da linguagem natural cujo contexto histórico e cultural, geralmente, vincula uma semântica à sintaxe.

3.1.1 Linguagem da lógica proposicional

A linguagem estabelece as regras de formação das fórmulas. Portanto, sequências que violam essas regras não são consideradas “falsas” (um conceito semântico), mas sim malformadas.

O *alfabeto* é o conjunto dos símbolos que compõem o sistema lógico. Os símbolos da linguagem da lógica proposicional são:

- Variáveis proposicionais: são os elementos fundamentais da lógica, também chamadas de fórmulas atômicas, e, normalmente, são representadas por letras do alfabeto como $X, X_1, X_2, Y, Z, Z_2^{p_1}$;
- Conectivos lógicos: símbolos que permitem a construção e ampliação de fórmulas a partir de outras. Os conectivos podem ser unários que são aplicados a uma única fórmula, binários que são aplicados a duas fórmulas e até mesmo n -ários que são aplicados a n fórmulas, com $n \in \mathbb{N}$. Por exemplo, o conectivo unário $\neg$ (negação) é utilizado como $(\neg Z)$, por outro lado, usa-se o conectivo binário $\wedge$ (conjunção) como $(Z_1 \wedge Z_2)$;
- Delimitadores: são os parênteses esquerdo “(” e direito “)” que servem para evitar ambiguidades.

Considerando $\Box$ um conectivo unário qualquer e $\Diamond$ um conectivo n -ário qualquer, as regras gramaticais para a produção de *fórmulas* com significado, no contexto da lógica proposicional, são:

1. Variáveis proposicionais são fórmulas;
2. Se Z é uma fórmula, $(\Box Z)$ é uma fórmula;
3. Se $Z_1, \dots, Z_n$ são fórmulas, então $(Z_1 \Diamond \dots \Diamond Z_n)$ é uma fórmula;
4. Não há outras fórmulas além das obtidas pelo uso das regras 1 a 3.

As fórmulas são representadas neste trabalho pelas letras gregas: φ e ψ e a omissão de parênteses é feita quando não cria ambiguidade na fórmula.

Para um detalhamento da construção da lógica proposicional feita até aqui, sugere-se a leitura do livro de SANTOS FAJARDO (2023).

3.1.2 Semântica da lógica proposicional

A *semântica* de uma lógica proposicional **Val** é o conjunto de valorações v , funções que atribuem um valor de um conjunto verdade $\mathcal{V}$ — conjunto de significados possíveis para v — para cada fórmula dentro da linguagem lógica $\mathcal{L}$, isto é, $v : \mathcal{L} \rightarrow \mathcal{V}$. E os valores

de “verdade absoluta” são os que pertencem ao conjunto $D \subset \mathcal{V}$, conjunto de valores que preservam a validade de sentenças em um sistema lógico. Avaliar uma fórmula $\varphi \in \mathcal{L}$ a partir de uma valoração v é o mesmo que calcular $v(\varphi)$.

As lógicas proposicionais diferem-se em conectivos lógicos e valorações, isto é, as lógicas podem possuir conjunto de fórmulas $\mathcal{L}$, conjunto verdade $\mathcal{V}$ e/ou conjunto de verdades designadas D diferentes.

3.2 Lógica proposicional clássica

Para a lógica proposicional clássica (CPL), que estende a lógica aristotélica, o conjunto de conectivos da linguagem é formado por:

- $\neg$ negação (não)
- $\wedge$ conjunção (e)
- $\vee$ disjunção (ou)
- $\rightarrow$ implicação (se ... então)
- $\leftrightarrow$ equivalência (se, e somente se)

Temos que, $\mathcal{V}_{CPL} = \{0, 1\}$, $D_{CPL} = \{1\}$ e a valoração $v : \mathcal{L}_{CPL} \rightarrow \mathcal{V}_{CPL}$, para fórmulas $\varphi, \psi \in \mathcal{L}_{CPL}$, é definida por:

$$\begin{aligned} v(\neg\varphi) &= 1 - v(\varphi); \\ v(\varphi \vee \psi) &= \max(v(\varphi), v(\psi)). \end{aligned}$$

Definindo a construção de todos os conectivos da lógica proposicional a partir dos conectivos $\neg$, e $\vee$, obtém-se:

$$\begin{aligned} (\varphi \wedge \psi) &:= \neg(\neg\varphi \vee \neg\psi) & v(\varphi \wedge \psi) &= \min(v(\varphi), v(\psi)) \\ (\varphi \rightarrow \psi) &:= \neg\varphi \vee \psi & v(\varphi \rightarrow \psi) &= \min(1, 1 - v(\varphi) + v(\psi)) \\ (\varphi \leftrightarrow \psi) &:= (\varphi \rightarrow \psi) \wedge (\psi \rightarrow \varphi) & v(\varphi \leftrightarrow \psi) &= 1 - |v(\varphi) - v(\psi)| \end{aligned}$$

A convenção adotada para omissão de parênteses é a de que $\neg$ tem precedência sobre $\wedge$ e $\vee$, que possuem precedência sobre $\rightarrow$ e $\leftrightarrow$.

3.3 Lógica de Łukasiewicz infinitamente-valorada

Para a lógica de Łukasiewicz infinitamente-valorada ($\mathbb{L}_\infty$), o conjunto de conectivos da linguagem é formado por:

- $\neg$ negação

$\odot$ conjunção

$\oplus$ disjunção

$\vee$ máximo

$\wedge$ mínimo

$\rightarrow$ implicação

$\leftrightarrow$ equivalência (bi-implicação)

Temos que, $\mathcal{V}_{\mathcal{L}_{\infty}} = [0, 1]$, $\mathcal{D}_{\mathcal{L}_{\infty}} = \{1\}$ e a valoração $v : \mathcal{L}_{\mathcal{L}_{\infty}} \rightarrow \mathcal{V}_{\mathcal{L}_{\infty}}$, para fórmulas $\varphi, \psi \in \mathcal{L}_{\mathcal{L}_{\infty}}$, é definida por:

$$\begin{aligned} v(\neg\varphi) &= 1 - v(\varphi); \\ v(\varphi \oplus \psi) &= \min(1, v(\varphi) + v(\psi)). \end{aligned}$$

E, derivando dos dois conectivos primários, temos:

$$\begin{aligned} (\varphi \odot \psi) &:= \neg(\neg\varphi \oplus \neg\psi) & v(\varphi \odot \psi) &= \max(0, v(\varphi) + v(\psi) - 1) \\ (\varphi \vee \psi) &:= \neg(\neg\varphi \oplus \psi) \oplus \psi & v(\varphi \vee \psi) &= \max(v(\varphi), v(\psi)) \\ (\varphi \wedge \psi) &:= \neg(\neg\varphi \vee \neg\psi) & v(\varphi \wedge \psi) &= \min(v(\varphi), v(\psi)) \\ (\varphi \rightarrow \psi) &:= \neg\varphi \oplus \psi & v(\varphi \rightarrow \psi) &= \min(1, 1 - v(\varphi) + v(\psi)) \\ (\varphi \leftrightarrow \psi) &:= (\varphi \rightarrow \psi) \wedge (\psi \rightarrow \varphi) & v(\varphi \leftrightarrow \psi) &= 1 - |v(\varphi) - v(\psi)| \end{aligned}$$

Seja X uma variável proposicional, então, $v(X \odot \neg X) = 0$, para qualquer $v \in \mathbf{Val}$, a constante $\mathbf{0}$ é definida por $X \odot \neg X$ e a constante $\mathbf{1}$ por $\neg\mathbf{0}$.

É importante notar que $v(\varphi \rightarrow \psi) = 1$ se, e somente se, $v(\varphi) \leq v(\psi)$ e, similarmente, $v(\varphi \leftrightarrow \psi) = 1$ se, e somente se, $v(\varphi) = v(\psi)$.

Definimos $0\varphi := \mathbf{0}$ e $n\varphi := \varphi \oplus \dots \oplus \varphi$, n vezes, para $n \in \mathbb{N}^*$.

Para a omissão de parênteses, a convenção adotada é a de que $\neg$ tem precedência sobre $\oplus$ e $\odot$, que possuem precedência sobre $\vee$ e $\wedge$, que possuem precedência sobre $\rightarrow$ e $\leftrightarrow$.

3.4 Semântica módulo satisfatibilidade

É dito que uma valoração $v \in \mathbf{Val}$ *satisfaz* uma fórmula $\varphi \in \mathcal{L}$ se $v(\varphi) \in \mathcal{D}$ e que *satisfaz* um conjunto de fórmulas $\Gamma \subseteq \mathcal{L}$ se satisfaz todas as fórmulas $\varphi \in \Gamma$, neste caso denotamos $v \models \varphi$ e $v \models \Gamma$, respectivamente. Uma fórmula ou conjunto de fórmulas é *satisfatível* se existe alguma valoração que a satisfaz e, caso o contrário, é *insatisfatível*.

A relação de consequência, denotada por $\models$, é tal que, para $\varphi \in \mathcal{L}$ e para $\Gamma \subseteq \mathcal{L}$, $\Gamma \models \varphi$ se, para toda valoração $v \in \mathbf{Val}$ que valha $v \models \Gamma$ temos que também vale $v \models \varphi$. Ou seja, se

$\Gamma \models \varphi$, então para as valorações em que Γ é satisfatível, φ também é satisfatível.

Para L_∞ , uma fórmula $\varphi \in \mathcal{L}_{L_\infty}$ é satisfatível se $\exists v \in \mathbf{Val}$ tal que $v(\varphi) = 1$. E, analogamente, um conjunto de fórmulas $\Gamma \subseteq \mathcal{L}_{L_\infty}$ é satisfatível se $\exists v \in \mathbf{Val}$ tal que, para qualquer $\psi \in \Gamma$, $v(\psi) = 1$.

De acordo com [PRETO \(2022\)](#), sendo $\Phi \subseteq \mathcal{L}$ um conjunto de fórmulas, a *semântica módulo satisfatibilidade* é o conjunto

$$\mathbf{Val}_\Phi = \{v \in \mathbf{Val} \mid v \models \Phi\}.$$

Isto é, a semântica módulo satisfatibilidade é uma restrição do conjunto $\mathbf{Val}$ e contém somente as valorações que satisfazem o conjunto de fórmulas Φ .

Para L_∞ , a semântica módulo satisfatibilidade é o conjunto que contém todos as valorações $v \in \mathbf{Val}$ tal que, $\forall \varphi \in \Phi$ vale que $v(\varphi) = 1$.

3.5 Representação módulo satisfatibilidade

A *representação módulo satisfatibilidade* permite-nos representar funções racionais de McNaughton em L_∞ ([PRETO e FINGER, 2022a](#); [PRETO e FINGER, 2020](#)). Uma função racional de McNaughton f , como descrita na Seção 2.3, é representada por um par $\langle \varphi, \Phi \rangle$, onde φ é a fórmula que assume valores equivalentes aos de $f(X)$, $\forall X \in [0, 1]^n$, a partir das valorações definidas pelo conjunto de fórmulas Φ na semântica módulo satisfatibilidade $\mathbf{Val}_\Phi$. É dito pelo **Teorema 3** de [PRETO \(2022\)](#) que: sendo $f : [0, 1]^n \rightarrow [0, 1]$ uma função racional de McNaughton multivariável. Então, f pode ser representada em L_∞ -MODSAT.

Para elucidar a representação módulo satisfatibilidade, faz-se necessário o enunciado de duas definições.

Definição 3.1. Seja φ uma fórmula e Φ um conjunto de fórmulas. Dizemos que o conjunto de variáveis proposicionais $\mathbf{X}^T = [X_1, X_2, \dots, X_n]$ *determina φ módulo Φ -satisfatível* se:

- Para qualquer $[x_1, \dots, x_n] \in [0, 1]^n$, existe ao menos uma valoração $v \in \mathbf{Val}_\Phi$, tal que $v(X_j) = x_j$, para $j = 1, \dots, n$;
- para qualquer par de valorações $v, v' \in \mathbf{Val}_\Phi$ tais que $v(X_j) = v'(X_j)$, para $j = 1, \dots, n$, temos que $v(\varphi) = v'(\varphi)$ — i.e. valorações em $\mathbf{Val}_\Phi$ são verofuncionais nas variáveis em $\mathbf{X}$.

O primeiro item garante que o conjunto de restrições Φ não seja tão restritivo ao ponto de impedir que as valorações atribuam valores reais no intervalo $[0, 1]$ para as variáveis X_j , com $j = 1, \dots, n$. O segundo item garante que o par funcione como uma função, isto é, atribua somente um valor de saída para cada entrada possível.

Definição 3.2. Seja $f : [0, 1]^n \rightarrow [0, 1]$ uma função e $\langle \varphi, \Phi \rangle$ um par onde φ é uma fórmula e Φ é um conjunto de fórmulas. Dizemos que φ *representa f módulo Φ -satisfatível* ou que $\langle \varphi, \Phi \rangle$ *representa f (no sistema L_∞ -MODSAT)* se:

- $\mathbf{X}$ determina φ módulo Φ -satisfatível;

- $f(v(X_1), \dots, v(X_n)) = v(\varphi)$, para toda $v \in \mathbf{Val}_\Phi$.

A primeira definição garante o domínio e o comportamento do par $\langle \varphi, \Phi \rangle$, a segunda definição garante que os valores da fórmula φ sejam equivalentes aos valores da imagem de f .

Exemplo 3.1. Toda função constante com valor a/b , com $a \in \mathbb{N}$ e $b \in \mathbb{N}^*$, pode ser representada pelo par $\langle \varphi, \Phi \rangle$, em que

$$\begin{aligned}\varphi &= \left\{ aZ_{\frac{1}{b}} \right\}, \\ \Phi &= \left\{ Z_{\frac{1}{b}} \leftrightarrow \neg(b-1)Z_{\frac{1}{b}} \right\}.\end{aligned}$$

Exemplo 3.2. Considere a função $f : [0, 1] \rightarrow [0, 1]$ que calcula $f(x_1) = x_1/3 + 1/2$, f é representada no sistema $\mathbb{L}_\infty\text{-MODSAT}$ pelo par $\langle \varphi, \Phi \rangle$ em que

$$\begin{aligned}\varphi &= \left\{ Z_1 \oplus Z_{\frac{1}{2}} \right\}, \\ \Phi &= \left\{ Z_1 \oplus Z_1 \oplus Z_1 \leftrightarrow X_1, \quad Z_{\frac{1}{3}} \leftrightarrow \neg(Z_{\frac{1}{3}} \oplus Z_{\frac{1}{3}}), \quad Z_1 \rightarrow Z_{\frac{1}{3}}, \quad Z_{\frac{1}{2}} \leftrightarrow \neg Z_{\frac{1}{2}} \right\}.\end{aligned}$$

Pela primeira fórmula de Φ , a variável proposicional X_1 é equivalente a três vezes o valor da variável proposicional Z_1 , a constante $1/3$ é definida pela segunda fórmula, pela terceira fórmula $Z_1 \leq 1/3$ e a constante $1/2$ é definida pela quarta fórmula. Por fim, a fórmula φ é equivalente à soma de $x_1/3 + 1/2$.

3.6 Algoritmos de representação MODSAT

3.6.1 Formato regional

O Algoritmo **NN2PWL**, que será abordado, utiliza a representação de funções de McNaughton no formato regional. Uma função racional de McNaughton $f : [0, 1]^n \rightarrow [0, 1]$ está codificada no *formato regional fechado* se é dada por m (não necessariamente distintas) partes lineares na forma

$$p_i(X) = \gamma_{i0} + \gamma_{i1}x_1 + \dots + \gamma_{in}x_n,$$

onde $X^T = [x_1, \dots, x_n] \in [0, 1]^n$, $\gamma_{ij} \in \mathbb{Q}$ e $i \in \{1, \dots, m\}$, tal que cada p_i é idêntico a f sobre o poliedro $\Omega_i \subseteq [0, 1]^n$, chamado de *região*. As regiões são determinadas pela intersecção finita dos semi-espacos gerados pelas inequações lineares em:

$$\Omega_i = \left\{ X \in [0, 1]^n \mid \omega_{j0} + \omega_{j1}x_1 + \dots + \omega_{jn}x_n \geq 0, \quad j \in \{1, \dots, \lambda_{\Omega_i}\} \right\}.$$

Tal configuração de partes lineares e regiões satisfazem as seguintes propriedades:

- $\bigcup_{i=1}^m \Omega_i = [0, 1]^n$;
- $\Omega_{i'} \cap \Omega_{i''} = \emptyset$, para $i' \neq i''$;

- A *propriedade do reticulado*: para $i \neq j$, existe um k tal que a parte linear p_i está acima da parte linear p_k sobre a região Ω_i , e a parte linear p_k está acima da parte linear p_j sobre a região Ω_j .

Sendo que dizer que uma parte linear a está acima de outra parte linear b sobre o conjunto Ω é o mesmo que dizer $a(X) \geq b(X)$, para todo $X \in \Omega$.

A propriedade do reticulado permite representar funções racionais de McNaughton baseadas em operações de máximos e mínimos (PRETO e FINGER, 2025). Isto é, sendo $f_{\Omega_j} : [0, 1]^n \rightarrow [0, 1]$ dada por

$$f_{\Omega_j}(X) = \min \left\{ p_k(X) \mid p_k \text{ está acima de } p_j \text{ sobre } \Omega_j \right\},$$

temos que

$$f(X) = \max \left\{ f_{\Omega_j}(X) \mid j \in \{1, \dots, m\} \right\}.$$

3.6.2 MLPs no formato regional

No artigo de PRETO e FINGER (2025), o Algoritmo **NN2PWL** para representar uma família de MLPs no formato regional é descrito. O algoritmo recebe de entrada um MLP com função de ativação ReLU e neurônios de saída truncados no intervalo $[0, 1]$. Isto é, um MLP com m neurônios de saída que computam, cada um, uma função racional de McNaughton $f_j : [0, 1]^n \rightarrow [0, 1]$, para $j \in \{1, \dots, m\}$. A saída do algoritmo é o conjunto de funções computadas pela rede neural no *formato pré-regional*.

O formato pré-regional satisfaz as duas propriedades do formato regional mas não há a garantia de que satisfaça a propriedade do reticulado. Entretanto, no artigo, os autores demonstram que as funções formatadas pelo algoritmo raramente não satisfazem a propriedade do reticulado — de 1100 redes neurais aplicadas no algoritmo durante os testes, somente uma rede neural não satisfaz a propriedade do reticulado (PRETO e FINGER, 2025).

O algoritmo descrito funciona em tempo exponencial em relação ao número de neurônios da rede neural, isto é, o incremento de neurônios incrementa exponencialmente o tempo de execução do algoritmo.

3.6.3 MLP em MODSAT

Para converter o conjunto de funções racionais de McNaughton, representadas no formato regional pelo Algoritmo **NN2PWL**, pode-se utilizar o Algoritmo **REPRESENT** em cada uma das funções (PRETO, 2022; PRETO e FINGER, 2023).

O Algoritmo **REPRESENT** recebe uma função racional de McNaughton no formato regional dada por seus coeficientes de suas partes lineares e as suas respectivas regiões, e retorna uma representação L_∞ -MODSAT $\langle \varphi, \Phi \rangle$ para a função.

O tempo de execução de **REPRESENT** é polinomial em relação ao tamanho da função de entrada.

Todavia, recentemente foi proposto um novo algoritmo (**NN2MODSAT**) para representar funções racionais de McNaughton no sistema $\mathbb{L}_\infty$ -MODSAT de modo computacionalmente mais eficaz em relação à combinação de **NN2PWL** e **REPRESENT**, pois descarta o passo intermediário de tradução do MLP para o formato regional, onde há maior consumo de recursos computacionais (**NN2PWL**), antes de estabelecer a representação $\mathbb{L}_\infty$ -MODSAT. O algoritmo em questão transforma, individualmente, os neurônios (que computam funções lineares) para a representação $\mathbb{L}_\infty$ -MODSAT e, então, combina estas representações codificando, na linguagem proposicional, o processo de computação através da rede neural (**Preto e Finger**). Na Tabela 5.1, pode-se comparar o tempo de execução dos dois métodos.

3.7 Resolução de satisfatibilidade e consequência lógica

Propriedades da rede neural podem ser modeladas em termos de verificação de fórmulas de $\mathbb{L}_\infty$.

A *alcançabilidade* nos diz se existe um cenário em que um neurônio de saída da rede neural atinge um determinado valor. Formalmente, seja $\langle \varphi, \Phi \rangle$ uma representação da função f , então dizemos que a saída de f atinge pelo menos o valor de a/b se, e somente se, a fórmula

$$\left(\bigwedge \Phi \right) \wedge \left(Z_{\frac{1}{b}} \leftrightarrow \neg(b-1)Z_{\frac{1}{b}} \right) \wedge \left(aZ_{\frac{1}{b}} \rightarrow \varphi \right) \quad (3.1)$$

é satisfatível, onde $(\bigwedge \Phi) = \psi_1 \wedge \psi_2 \wedge \dots \wedge \psi_n$, para $\psi_i \in \Phi$, com $i \in \{0, 1, \dots, n\}$ (**PRETO e FINGER, 2022b**). A fórmula (3.1) pode ser dividida em três partes na seguinte forma:

- $(\bigwedge \Phi)$: esta parte ser satisfatível significa que estamos lidando com a semântica MODSAT, ou seja, todas as fórmulas do conjunto Φ são satisfatíveis;
- $(Z_{\frac{1}{b}} \leftrightarrow \neg(b-1)Z_{\frac{1}{b}})$: a satisfatibilidade dessa fórmula define, como no Exemplo 3.1, $v(Z_{\frac{1}{b}}) = 1/b$, $\forall v$ tal que $v \models (Z_{\frac{1}{b}} \leftrightarrow \neg(b-1)Z_{\frac{1}{b}})$;
- $(aZ_{\frac{1}{b}} \rightarrow \varphi)$: sendo satisfatível, o lado direito da implicação é maior do que ou igual ao lado esquerdo, isto é, a fórmula φ , que representa a saída da função f , é maior do que ou igual a constante a/b .

Em resumo, verificar se uma função f atinge pelo menos um valor a/b significa verificar se (3.1) é satisfatível.

A *consequência lógica* é utilizada para verificar se, quando o conjunto de premissas é satisfatível, o conjunto de conclusões é satisfatível. Formalmente, seja Γ_p o conjunto de premissas e Γ_c o conjunto de conclusões, dizemos que Γ_c é consequência lógica de Γ_p , isto é, $\Gamma_p \models \Gamma_c$ se

$$\forall v \in \mathbf{Val} \text{ é tal que } \Gamma_p \text{ é satisfatível por } v, \text{ então } \Gamma_c \text{ é satisfatível por } v.$$

Para resolver os problemas de consequência lógica e de satisfatibilidade há diversos

algoritmos. Neste trabalho, utilizamos o Algoritmo **LUKASOL** que reduz esses problemas para problemas de programação linear inteira mista (MILP), a partir da biblioteca SCIP.¹

Sumarizando o capítulo, é possível representar funções racionais de McNaughton e, consequentemente, redes neurais que atendem os critérios do Teorema 2.1, no sistema $\mathbb{L}_\infty$ -MODSAT. E, posteriormente, pode-se utilizar as ferramentas lógicas para provar-se propriedades da rede neural utilizando, por exemplo, a alcançabilidade.

¹ Para mais detalhes sobre os métodos empregados, ver HÄHNLE (1994), BOFILL *et al.* (2019) e PRETO, MANYÀ *et al.* (2023)

Capítulo 4

Treinamento das redes neurais

Este capítulo visa trazer ao leitor detalhes do treinamento de duas redes neurais simples para a tarefa de detecção de intrusão em redes digitais. Para tal, inicialmente, o conjunto de dados *CIC-IDS2017* e a sua manipulação são detalhados na Seção 4.1. Em seguida, a estrutura das redes neurais, cujas saídas computam exatamente funções racionais de McNaughton, é definida na Seção 4.2. E, por fim, as estratégias de treinamento empregadas e os relatórios de avaliação das redes neurais, a partir de métricas explicitadas no texto, são discutidas na Seção 4.3.

4.1 Preparação do *CIC-IDS2017*

Para o treinamento das redes neurais, faz-se necessária uma série de tratamentos no conjunto de dados *CIC-IDS2017*. Em [HOOGLAND \(2018\)](#), o conjunto foi limpo, processo no qual os *metadados* foram removidos. Tal remoção é necessária, pois os metadados trazem informações sobre o conjunto de dados e não sobre o tráfego de rede.

Em seguida, os dados duplicados e nulos foram removidos, pois não trazem informações novas para o treinamento da rede neural. A Tabela 4.1 fornece o número de registros para cada tipo de tráfego após a limpeza.

O objetivo primário deste trabalho é a análise formal de redes neurais, envolvendo tanto a proposta e a formalização de propriedades quanto a interpretação de resultados de análises empíricas. Portanto, apenas cinco principais características do conjunto de dados foram selecionadas para o treinamento da rede, com o intuito de reduzir o seu tamanho. Essa decisão se justifica pelo fato de que a representação *MODSAT* de redes neurais é proporcional ao número de neurônios, de modo que arquiteturas maiores implicariam um aumento na complexidade dos modelos analisados. Assim, a redução do número de características contribui para mitigar problemas de escalonamento nos experimentos.

As principais características foram selecionadas de acordo com o critério de *análise de variância* (ANOVA). Basicamente, as cinco características de maior correlação estatística linear com os rótulos foram selecionadas. Após a limpeza e a seleção, os valores numéricos foram normalizados para o intervalo $[0, 1]$. Esse processo é fundamental, pois o objetivo é

Rótulo	Número de registros
Benigno	1.895.314
DoS Hulk	172.846
DDoS	128.014
DoS GoldenEye	10.286
FTP-Patator	5.931
DoS slowloris	5.385
DoS slowhttptest	5.228
SSH-Patator	3.219
PortScan	1.956
Ataque Web: Força Bruta	1.470
Bot	1.437
Ataque Web: XSS	652
Infiltração	36
Ataque Web: Injeção de SQL	21
Heartbleed	11
Total	2.231.806

Tabela 4.1: Número de registros de tráfego de rede por rótulo (CIC-IDS2017 tratado)

treinar um MLP que represente uma função racional de McNaughton $f : [0, 1]^n \rightarrow [0, 1]$.

Por fim, antes do treinamento, o conjunto de dados foi dividido em três partes. O conjunto de treinamento, utilizado para realizar o aprendizado de máquina supervisionado, corresponde a 80% do conjunto original; o conjunto de validação, utilizado durante o treinamento para verificar o desempenho da rede treinada, corresponde a 15%; e o conjunto de teste, utilizado após o treinamento para verificar o funcionamento da rede neural, corresponde a 5%.

4.2 Estrutura dos MLPs

A estrutura dos MLPs treinados é representada pela Figura 4.1 e consiste em:

- Camada de entrada de dimensão 5, que recebe as características selecionadas do *CIC-IDS2017*;
- Duas camadas ocultas, cada uma com 16 neurônios, e com função de ativação ReLU;
- Camada de saída de dimensão 15, na qual cada unidade de saída representa uma classificação possível para o tráfego inserido na entrada. Na camada de saída, a função identidade truncada $\tau : \mathbb{R} \rightarrow [0, 1]$, $\tau(x) = \min(1, \max(x, 0))$, é aplicada para truncar os valores de saída.

Cada neurônio j , da camada de saída, representa uma função racional de McNaughton $f_j : [0, 1]^5 \rightarrow [0, 1]$.

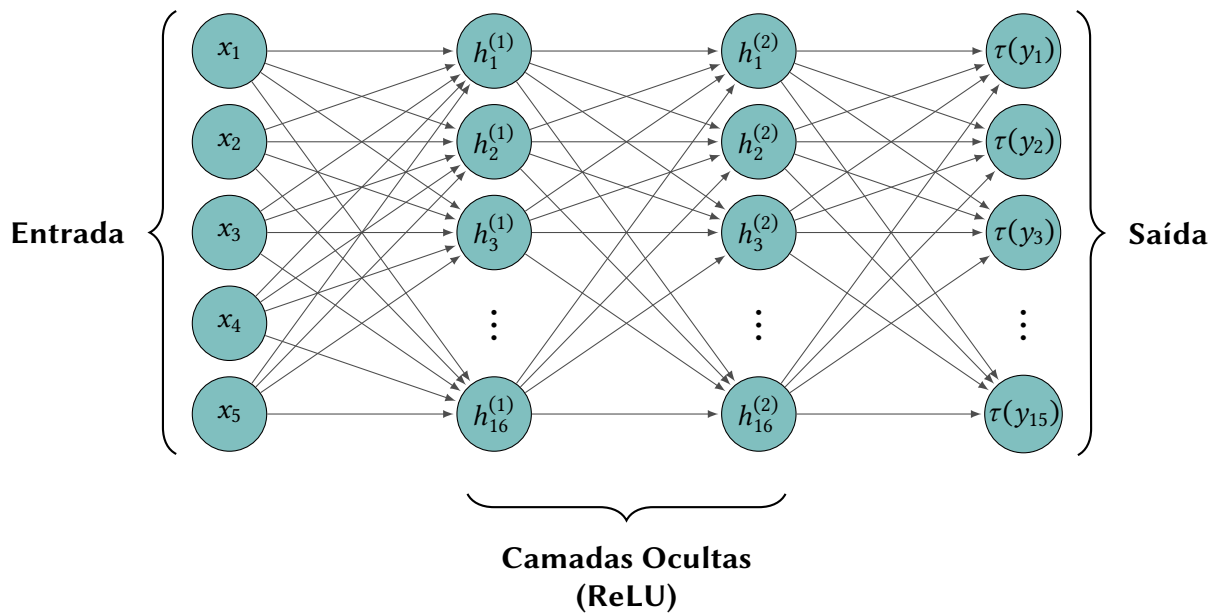


Figura 4.1: Estrutura dos MLPs treinados

4.3 Treinamento dos MLPs

O otimizador Adam (*Adaptive Moment Estimation*) da biblioteca *PyTorch* foi o método escolhido para otimizar os parâmetros da rede neural a partir do algoritmo de retropropagação (2.2). A escolha deve-se ao fato de o otimizador *Adam* combinar as vantagens de dois otimizadores populares — *AdaGrad* e *RMSProp* — e sua implementação ser simples e eficiente (KINGMA e BA, 2017).

A função de custo escolhida, responsável por penalizar a rede neural com base na divergência entre a previsão e o alvo, é a Perda de Entropia Cruzada (*Cross Entropy Loss*), amplamente utilizada para a classificação multiclasse.

Duas estratégias de treinamento foram empregadas para gerar duas redes neurais diferentes para a análise formal.

A primeira estratégia consiste em treinar a rede neural utilizando todos os dados do conjunto de treinamento de uma vez, estratégia conhecida como *full batch*, nessa estratégia o erro médio é calculado de todo o conjunto de dados de treinamento e os pesos são atualizados somente uma vez por época — passagem completa pelo conjunto de dados. Chamaremos o MLP treinado pela estratégia de *full batch* como *MLP-0*.

Para a segunda estratégia, o conjunto de dados de treinamento é dividido em pequenos pacotes (*mini-batch*) e os pesos são atualizados diversas vezes por época. Chamaremos o MLP treinado pela estratégia de *mini-batches* como *MLP-1*.

Após o treinamento, os desempenhos de ambas as redes neurais foram avaliados a partir da aplicação do conjunto de teste. Para cada classe, a classificação realizada pela rede neural pode ser dividida em quatro possibilidades, para a classe de tráfego benigno, por exemplo, a matriz de confusão é definida na Tabela 4.2.

	Positivo real	Negativo real
Previsão positiva	Verdadeiro positivo (VP): tráfego de rede classificado corretamente como benigno.	Falso positivo (FP): tráfego de rede que não é benigno classificado incorretamente como benigno
Previsão negativa	Falso negativo (FN): tráfego de rede benigno classificado incorretamente como não benigno.	Verdadeiro negativo (VN): tráfego de rede não benigno classificado corretamente como não benigno.

Tabela 4.2: Matriz de confusão

As métricas de avaliação aplicadas, para o tráfego benigno são:

- **Precisão:** mede o percentual de acerto ao classificar o tráfego como benigno. É calculada pela expressão

$$\frac{VP}{VP + FP};$$

- **Revocação (*recall*):** mede o percentual de tráfegos categorizados como benignos em relação ao total de tráfegos verdadeiramente benignos. É calculado pela expressão

$$\frac{VP}{VP + FN};$$

- **Pontuação F1:** é a média harmônica entre a precisão e a revocação. É preferível para conjuntos de dados desbalanceados, como o caso do *CIC-IDS2017*, pois a pontuação será de acordo com o equilíbrio entre precisão e revocação, ou seja, a pontuação máxima é obtida quando as métricas de precisão e de revocação são iguais. Calculada pela expressão

$$\frac{2 \cdot \text{precisão} \cdot \text{revocação}}{\text{precisão} + \text{revocação}};$$

- **Suporte:** É o número de elementos da classe no conjunto de teste;
- **Acurácia:** mede o percentual de acertos da classificação. É a proporção entre o número de tráfegos benignos e não benignos classificados corretamente em relação ao total de tráfegos.

Ao analisar os resultados das Tabelas 4.3 e 4.4, para o conjunto de teste, pode-se perceber que ambas as redes não classificaram corretamente nenhum registro com exceção das classes *Benigno* e *DDoS*. Surge, então, a partir dos relatórios, a hipótese de que ambas as redes não aprenderam a classificar classes diferentes de *Benigno* e *DDoS* por conta do desbalanceamento severo entre classes aliado à arquitetura simplificada. No próximo capítulo, com a análise formal, poderemos verificar a hipótese e novas propriedades.

É importante ressaltar que os relatórios de classificação das Tabelas 4.3 e 4.4 foram realizados com um conjunto de teste e, por isso, há classes que não possuem registros suficientes para uma avaliação correta. Como as classes: *Heartbleed*, *Ataque Web: Injeção de SQL* e *Infiltração* que possuem 0, 1 e 2 registros, respectivamente.

#	Classe	Precisão	Revocação	Pontuação F1	Suporte
0	Benigno	92%	99%	95%	94.766
1	Bot	0%	0%	0%	72
2	FTP-Patator	0%	0%	0%	297
3	SSH-Patator	0%	0%	0%	161
4	DDoS	45%	64%	53%	6.401
5	DoS slowloris	0%	0%	0%	269
6	DoS slowhttptest	0%	0%	0%	262
7	DoS Hulk	0%	0%	0%	8.642
8	DoS GoldenEye	0%	0%	0%	514
9	Heartbleed	0%	0%	0%	0
10	Infiltração	0%	0%	0%	2
11	PortScan	0%	0%	0%	98
12	Ataque Web: Força Bruta	0%	0%	0%	74
13	Ataque Web: XSS	0%	0%	0%	32
14	Ataque Web: Injeção de SQL	0%	0%	0%	1
Acurácia				88%	111.591

Tabela 4.3: Relatório de classificação para o MLP-0

#	Classe	Precisão	Revocação	Pontuação F1	Suporte
0	Benigno	90%	100%	95%	94.766
1	Bot	0%	0%	0%	72
2	FTP-Patator	0%	0%	0%	297
3	SSH-Patator	0%	0%	0%	161
4	DDoS	61%	63%	62%	6.401
5	DoS slowloris	0%	0%	0%	269
6	DoS slowhttpstest	0%	0%	0%	262
7	DoS Hulk	0%	0%	0%	8.642
8	DoS GoldenEye	0%	0%	0%	514
9	Heartbleed	0%	0%	0%	0
10	Infiltração	0%	0%	0%	2
11	PortScan	0%	0%	0%	98
12	Ataque Web: Força Bruta	0%	0%	0%	74
13	Ataque Web: XSS	0%	0%	0%	32
14	Ataque Web: Injeção de SQL	0%	0%	0%	1
Acurácia				89%	111.591

Tabela 4.4: Relatório de classificação para o MLP-1

Capítulo 5

Análise formal das redes neurais

Os detalhes da análise formal das redes neurais $MLP-0$ e $MLP-1$ são apresentados neste capítulo. Primeiro, as redes neurais são representadas em $\mathbb{L}_\infty$ -MODSAT, a partir dos algoritmos de representação MODSAT, na Seção 5.1. Em seguida, as propriedades, representadas por fórmulas de $\mathbb{L}_\infty$, são construídas na Seção 5.2. E, por fim, os resultados, gerados através de resolvidores de satisfatibilidade e provadores de teoremas para a lógica de Łukasiewicz, e interpretações da análise formal são discutidos na Seção 5.3.

5.1 Representação MODSAT

No início do desenvolvimento deste trabalho, utilizando uma rede neural preliminar, foi realizada a comparação entre os tempos de execução dos algoritmos de representação módulo satisfatibilidade. Inicialmente, utilizou-se o Algoritmo NN2PWL, para transformar a rede neural em uma função racional de McNaughton no formato regional, em combinação com o REPRESENT (PRETO, 2022; PRETO e FINGER, 2023), para representar a função na linguagem $\mathbb{L}_\infty$. Por outro lado, o Algoritmo NN2MODSAT (Preto e Finger), que não utiliza o formato regional, foi aplicado na mesma rede neural. Na Tabela 5.1, pode-se comparar o tempo de execução entre ambas as abordagens.

	NN2PWL e REPRESENT	NN2MODSAT
Tempo de execução	37 horas, 21 minutos e 47 segundos.	3 segundos.

Tabela 5.1: Comparação entre abordagens de representação MODSAT

O computador utilizado para realizar o teste do tempo de execução e todos os outros testes presentes neste trabalho possui as configurações descritas na Tabela 5.2.

Pela eficácia em tempo de execução e tamanho da representação, a abordagem baseada no Algoritmo NN2MODSAT foi escolhida para representar as redes neurais $MLP-0$ e $MLP-1$ no sistema $\mathbb{L}_\infty$ -MODSAT.

Componente	Especificação
Processador (CPU)	Intel(R) Core(TM) i5-6440HQ CPU @ 2.60GHz (4 núcleos)
Memória RAM	24 GB DDR4
Placa de Vídeo (GPU)	NVIDIA Quadro M2000M (4GB VRAM)
Armazenamento	SSD 480 GB SATA III (Kingston A400)
Sistema Operacional	Fedora Linux 42 (Workstation Edition)
Kernel	6.17.8-200.fc42.x86_64

Tabela 5.2: Especificações do ambiente computacional utilizado nos experimentos.

5.2 Construção de propriedades

Uma vez que representamos ambas as redes neurais no sistema $\mathbb{L}_\infty$ -MODSAT, podemos utilizar as representações para fazer testes de verificação de fórmulas que indicam alguma propriedade sobre as redes neurais.

A representação MODSAT da rede neural $MLP-i$, com $i \in \{0, 1\}$, é denotada pelos pares $\langle \varphi_{i,j}, \Phi_i \rangle$, para $j = 0, \dots, 14$, onde j representa uma unidade de saída da rede neural. O conjunto de restrição Φ_i é idêntico para todos os pares de sua respectiva rede neural. Todas as propriedades são testadas por meio de um provador de teoremas, isto é, um algoritmo que busca uma valoração que satisfaça a fórmula de $\mathbb{L}_\infty$ que modela a propriedade, para os casos das Propriedades 1 e 2, ou que verifica se a consequência lógica é válida, para os casos das Propriedades 3 e 4.

5.2.1 Propriedade 1: Alcançabilidade

A primeira propriedade analisada, denotada pela fórmula (5.1), tem como objetivo determinar se o neurônio de saída $\varphi_{i,j}$ pode atingir um nível de ativação superior ou igual a um limiar pré-determinado $a/b \in [0, 1]$ com $a \in \mathbb{N}$ e $b \in \mathbb{N}^*$.

A fórmula de alcançabilidade, definida em (3.1), é expressa por:

$$\left(\bigwedge \Phi_i \right) \wedge \left(Z_{\frac{1}{b}} \leftrightarrow \neg(b-1)Z_{\frac{1}{b}} \right) \wedge \left(aZ_{\frac{1}{b}} \rightarrow \varphi_{i,j} \right). \quad (5.1)$$

Se o solucionador retornar que (5.1) é satisfatível, conclui-se que existe pelo menos uma configuração de entrada capaz de excitar a classe j com intensidade mínima de a/b .

5.2.2 Propriedade 2: Saída selecionada

A segunda propriedade (5.2) modela a condição de classificação de uma classe específica. Diferentemente da propriedade anterior, que analisava a intensidade absoluta de um sinal, esta fórmula avalia a intensidade relativa entre as saídas.

$$\left(\bigwedge \Phi_i \right) \wedge \left(\bigwedge_{k \neq j} (\varphi_{i,k} \rightarrow \varphi_{i,j}) \right) \quad (5.2)$$

A diferença em relação à propriedade anterior é o termo $(\bigwedge_{k \neq j} (\varphi_{i,k} \rightarrow \varphi_{i,j}))$ que, quando satisfatível, impõe que, para todo neurônio de saída k distinto do neurônio alvo j , deve valer a relação $v(\varphi_{i,k}) \leq v(\varphi_{i,j})$. Em termos práticos, satisfazer esta propriedade equivale a constatar que existe uma configuração de entrada tal que a rede neural a classifica como a classe j , considerando possíveis empates.¹

5.2.3 Propriedade 3: Saída selecionada com limitação do complemento

A terceira propriedade analisada estabelece uma relação de consequência lógica. Seu objetivo é verificar a certeza da decisão da rede, impondo um teto de ativação para as classes não selecionadas.

Seja

$$\Gamma_1 = \left(\bigwedge \Phi_i \right) \wedge \left(\bigwedge_{k \neq j} (\varphi_{i,k} \rightarrow \varphi_{i,j}) \right) \wedge \left(Z_{\frac{1}{b}} \leftrightarrow \neg(b-1)Z_{\frac{1}{b}} \right), \quad (5.3)$$

$$\Gamma_2 = \bigwedge_{k \neq j} (\varphi_{i,k} \rightarrow aZ_{\frac{1}{b}}), \quad (5.4)$$

então, a Propriedade 3 é expressa por:

$$\Gamma_1 \models \Gamma_2. \quad (5.5)$$

O conjunto de premissas Γ_1 agrega as definições apresentadas nas propriedades anteriores: as valorações devem pertencer à semântica módulo satisfatibilidade, a definição da constante racional $Z_{\frac{1}{b}}$, e a condição de que a classe j seja a classificada (o termo $\bigwedge_{k \neq j} (\varphi_{i,k} \rightarrow \varphi_{i,j})$, conforme definido na Propriedade 2).

A consequência Γ_2 introduz uma restrição de segurança sobre os neurônios de saída não selecionados. A fórmula $\bigwedge_{k \neq j} (\varphi_{i,k} \rightarrow aZ_{\frac{1}{b}})$ determina que, dado que j foi a classe predita, a ativação de qualquer outra classe k não deve ultrapassar o limiar a/b . Validar essa propriedade potencializa a ideia de que, nas configurações de entrada classificadas como j , não há incerteza significativa, pois as demais classes são mantidas abaixo de um nível de ativação determinado.

Uma observação interessante é a de que, caso haja o interesse em testar essa propriedade utilizando satisfatibilidade, há uma aproximação do resultado pela seguinte fórmula:

¹ A fórmula é satisfatível em casos de empate, e, nesses casos, a classificação é feita através de um critério de desempate.

$$\left(\bigwedge \Phi_i\right) \wedge \left(\bigwedge_{k \neq j} (\varphi_{i,k} \rightarrow \varphi_{i,j})\right) \wedge \left(Z_{\frac{1}{b}} \leftrightarrow \neg(b-1)Z_{\frac{1}{b}}\right) \wedge \left(aZ_{\frac{1}{b}} \rightarrow \bigvee_{k \neq j} \varphi_{i,k}\right), \quad (5.6)$$

isto é, a fórmula é satisfatível se existe uma valoração tal que satisfaça Γ_1 e $v(\varphi_{i,k}) \geq a/b, \forall k \neq j$. Tal abordagem não é equivalente a testar a consequência lógica de (5.4), pois os casos em que $v(\varphi_{i,k}) = a/b$ satisfazem (5.6) que falseariam incorretamente a Propriedade 3.

5.2.4 Propriedade 4: Saída selecionada isoladamente

Considere as duas variáveis proposicionais Z_1 e Z_2 , temos que

$$v(\neg(Z_1 \rightarrow Z_2)) = 1 - \min(1, 1 - v(Z_1) + v(Z_2)),$$

e, sempre que $v(Z_1) \geq v(Z_2)$, vale que

$$v(\neg(Z_1 \rightarrow Z_2)) = v(Z_1) - v(Z_2). \quad (5.7)$$

A quarta propriedade também estabelece uma relação de consequência lógica. Seu objetivo, analogamente à Propriedade 3, é verificar a certeza da decisão da rede, verificando a distância entre o neurônio selecionado e os outros.

Seja

$$\Gamma_1 = \left(\bigwedge \Phi_i\right) \wedge \left(\bigwedge_{k \neq j} (\varphi_{i,k} \rightarrow \varphi_{i,j})\right) \wedge \left(Z_{\frac{1}{b}} \leftrightarrow \neg(b-1)Z_{\frac{1}{b}}\right), \quad (5.8)$$

$$\Gamma_3 = \bigwedge_{k \neq j} \left(aZ_{\frac{1}{b}} \rightarrow \neg(\varphi_{i,j} \rightarrow \varphi_{i,k})\right), \quad (5.9)$$

então, a Propriedade 4 é expressa por:

$$\Gamma_1 \models \Gamma_3. \quad (5.10)$$

O conjunto de premissas Γ_1 é o mesmo da Propriedade 3, e é fundamental para a Propriedade 4 pois o termo $\bigwedge_{k \neq j} (\varphi_{i,k} \rightarrow \varphi_{i,j})$ garante que $v(\varphi_{i,k}) \leq v(\varphi_{i,j})$ e, então, a diferença entre $v(\varphi_{i,j})$ e $v(\varphi_{i,k})$ pode ser expressa por (5.7).

A consequência Γ_3 introduz uma restrição de segurança sobre os neurônios de saída não selecionados. A fórmula $\bigwedge_{k \neq j} (aZ_{\frac{1}{b}} \rightarrow \neg(\varphi_{i,j} \rightarrow \varphi_{i,k}))$ determina que, dado que j foi a classe predita, a diferença entre a ativação do neurônio j e a ativação do neurônio k é maior do que ou igual ao limiar $a/b, \forall k \neq j$. Ou seja, a ativação do neurônio j está isolada das outras ativações em pelo menos a/b de distância. Validar essa propriedade potencializa a ideia de que, nas configurações de entrada classificadas como j , não há incerteza significativa, pois a classe j foi selecionada com margem.

E, modelando uma aproximação do teste, em termos de satisfatibilidade, temos

$$\left(\bigwedge \Phi_i \right) \wedge \left(\bigwedge_{k \neq j} (\varphi_{i,k} \rightarrow \varphi_{i,j}) \right) \wedge \left(Z_{\frac{1}{b}} \leftrightarrow \neg(b-1)Z_{\frac{1}{b}} \right) \wedge \left(\neg \left(\varphi_{i,j} \rightarrow \bigvee_{k \neq j} \varphi_{i,k} \right) \rightarrow aZ_{\frac{1}{b}} \right),$$

que é satisfatível somente se Γ_1 for satisfatível e se $v(\varphi_{i,j}) - \max(v(\varphi_{i,k})) \geq a/b$. O que também falseia incorretamente (5.10) para o caso em que $v(\varphi_{i,j}) - \max(v(\varphi_{i,k})) = a/b$.

5.3 Testes das propriedades

A partir do Algoritmo **LUKASOL**,² que utiliza a biblioteca SCIP para realizar a verificação formal de fórmulas do sistema $\mathbb{L}_\infty$, as quatro propriedades anteriores foram testadas para cada unidade de saída de ambas as redes neurais, sendo as duas primeiras testadas por *alcançabilidade* e as duas últimas por *consequência lógica*.

A Propriedade 1 foi testada para uma constante $a/b = 0,7$, um valor relativamente alto para o intervalo $[0, 1]$. A Propriedade 3 foi testada para um limiar de $a/b = 0,5$. E, por fim, a Propriedade 4 foi testada com três valores diferentes: 4.a com constante 0,2; 4.b com constante 0,1; e 4.c com constante 0,05.

5.3.1 MLP-0

Para os resultados da rede neural *MLP-0*, na Tabela 5.3, verificamos que as classes que satisfazem a Propriedade 1 são, somente, as classes *Bot*, *DDoS* e *DoS Hulk*. O que indica que as outras unidades de saída não são excitadas acima de 0,7 para nenhuma combinação de tráfego de rede das características selecionadas, portanto, o *MLP-0* não é uma rede neural de alta sensibilidade pois a maioria de suas unidades de saída não atingem 0,7.

Para a Propriedade 2, todas as categorias, com exceção de *Heartbleed* e *Ataque Web: XSS*, a satisfazem. Isto é, a rede neural aprendeu a classificá-las, negando a hipótese gerada pelo relatório da Tabela 4.3 de que a rede só havia aprendido a identificar *Benigno* e *DDoS*. Entretanto, é importante ressaltar que é possível que alguns rótulos sejam raramente preditos pela rede por conta do grande desbalanceamento no conjunto de dados *CIC-IDS2017*.

Para a Propriedade 3 (que depende da Propriedade 2), somente *DoS GoldenEye* entre as classes atingíveis — que satisfazem a Propriedade 2 — não possui a Propriedade 3 válida. Ou seja, para quase todos os rótulos que a rede neural aprendeu a classificar, o complemento está abaixo de um limiar, o que indica uma confiabilidade na identificação. Entretanto, a insatisfatibilidade da Propriedade 1 enfraquece esta propriedade, pois indica que a classe selecionada está sempre abaixo de 0,7 e, então, pode estar, também, abaixo de 0,5.

Por fim, nenhuma unidade de saída atende a Propriedade 4, que também depende da Propriedade 2, para nenhuma constante testada $a/b \in \{0,2; 0,1; 0,05\}$. Com essa informação, entende-se que, apesar de a rede neural *MLP-0* ser capaz de discriminar quase todas as classes do conjunto de treinamento, existem classificações que não atendem os critérios de margem de segurança.

² Disponível publicamente no repositório do *Github*: <https://github.com/spreto/luksol>

A interpretação dos resultados acontece sob a ótica de que somente cinco características foram selecionadas do conjunto de dados. Uma possibilidade de interpretação dos resultados é considerar que os tráfegos que acusam diferentes classes não diferem tanto em relação às características selecionadas, o que indicaria que os resultados de proximidade (Propriedades 3 e 4) dos valores de saída da rede são intrínsecos a esse tipo de previsão.

Por outro lado, se entendermos que as classes que representam ataques cibernéticos deveriam diferir fundamentalmente da classe *Benigno*, nossos resultados mostram que as redes treinadas ainda não fazem boas previsões para o problema abordado, dado que não satisfazem as Propriedades 3 e/ou 4 para nenhuma constante testada.

Uma terceira interpretação possível é a de que alguns perfis de ataque, de fato, deveriam ser acusados com valores de saída da rede neural próximos, como o perfil dos ataques de negação de serviço (*DDoS*, *DoS slowloris*, *DoS slowhttptest*, *DoS Hulk*, *DoS GoldenEye*), e outras classes são isoladas, inclusive a classe *Benigno*. Para esse caso, seria de grande importância o atendimento das Propriedades 3 e/ou 4 para as classes de tráfego não semelhantes.

Em suma, para o *MLP-0*, o baixo nível de atendimento da Propriedade 1 indica que a rede neural opera em valores relativamente baixos, o que corrobora para o atendimento da Propriedade 3 sem grandes méritos. E, por isso, para a implementação de uma tal rede neural para a detecção de intrusão, seria positivo aplicar outros métodos de treinamento para que a rede satisfaça todas as propriedades.

Notamos uma inconsistência nos testes para as Propriedades 4.a, 4.b e 4.c para as classes *Heartbleed* e *Ataque Web: XSS*. Como a Propriedade 2 aparece como uma premissa Γ_1 em (5.10), se a Propriedade 2 é insatisfatível então a consequência lógica deveria ser válida por vacuidade. Porém, o solucionador reportou consequências lógicas inválidas. Creditamos esse comportamento a uma instabilidade numérica do solucionador, **LUKASOL**, para problemas de otimização linear inteira mista (MILP), dada a grande quantidade de restrições lineares que são geradas para essas instâncias.

5.3.2 *MLP-1*

Para a rede neural *MLP-1*, na Tabela 5.4, verificamos que quase todas as classes satisfazem a Propriedade 1, com exceção de *DoS slowhttptest* e *DoS GoldenEye*, o que indica que o *MLP-1* é uma rede neural de alta sensibilidade pois existem configurações de entrada que fazem a maioria de suas unidades de saída atingirem pelo menos 0,7.

Em relação a Propriedade 2, todas as classes a satisfazem, o que indica que a rede neural aprendeu a classificar todas as classes do conjunto de entrada, como desejado. Todavia, todas as outras propriedades (3, 4.a, 4.b e 4.c) não são atendidas, indicando que não há uma margem de segurança nas classificações da rede neural.

Dessa forma, conclui-se que a rede neural opera com valores altos, pois a Propriedade 1 é atendida em quase todos os casos e a Propriedade 3 nunca é atendida, e, também, que a rede neural não seleciona unidades de saída com uma margem de segurança, isto é, excitando o neurônio selecionado de forma mais intensa que os outros.

Conclui-se, então, que, de modo análogo ao *MLP-0*, o *MLP-1* carece da satisfação de

algumas propriedades fundamentais, se não considerarmos a primeira abordagem: os tráfegos de rede gerados pelas classes são, em até certo nível, semelhantes.

#	Classe	Propriedades					
		1	2	3	4.a	4.b	4.c
0	Benigno	unSAT	SAT	Valid	inValid	inValid	inValid
1	Bot	SAT	SAT	Valid	inValid	inValid	inValid
2	FTP-Patator	unSAT	SAT	Valid	inValid	inValid	inValid
3	SSH-Patator	unSAT	SAT	Valid	inValid	inValid	inValid
4	DDoS	SAT	SAT	Valid	inValid	inValid	inValid
5	DoS slowloris	unSAT	SAT	Valid	inValid	inValid	inValid
6	DoS slowhttpstest	unSAT	SAT	Valid	inValid	inValid	inValid
7	DoS Hulk	SAT	SAT	Valid	inValid	inValid	inValid
8	DoS GoldenEye	unSAT	SAT	inValid	inValid	inValid	inValid
9	Heartbleed*	unSAT	unSAT	Valid	inValid	inValid	inValid
10	Infiltração	unSAT	SAT	Valid	inValid	inValid	inValid
11	PortScan	unSAT	SAT	Valid	inValid	inValid	inValid
12	Ataque Web: Força Bruta	unSAT	SAT	Valid	inValid	inValid	inValid
13	Ataque Web: XSS*	unSAT	unSAT	Valid	inValid	inValid	inValid
14	Ataque Web: Injeção de SQL	unSAT	SAT	Valid	inValid	inValid	inValid

Legenda: unSAT: Insatisfatível; SAT: Satisfatível; inValid: Consequência inválida; Valid: Consequência válida.

Tabela 5.3: Resultados dos testes de verificação formal para o MLP-0

#	Classe	Propriedades					
		1	2	3	4.a	4.b	4.c
0	Benigno	SAT	SAT	inValid	inValid	inValid	inValid
1	Bot	SAT	SAT	inValid	inValid	inValid	inValid
2	FTP-Patator	SAT	SAT	inValid	inValid	inValid	inValid
3	SSH-Patator	SAT	SAT	inValid	inValid	inValid	inValid
4	DDoS	SAT	SAT	inValid	inValid	inValid	inValid
5	DoS slowloris	SAT	SAT	inValid	inValid	inValid	inValid
6	DoS slowhttptest	unSAT	SAT	inValid	inValid	inValid	inValid
7	DoS Hulk	SAT	SAT	inValid	inValid	inValid	inValid
8	DoS GoldenEye	unSAT	SAT	inValid	inValid	inValid	inValid
9	Heartbleed	SAT	SAT	inValid	inValid	inValid	inValid
10	Infiltração	SAT	SAT	inValid	inValid	inValid	inValid
11	PortScan	SAT	SAT	inValid	inValid	inValid	inValid
12	Ataque Web: Força Bruta	SAT	SAT	inValid	inValid	inValid	inValid
13	Ataque Web: XSS	SAT	SAT	inValid	inValid	inValid	inValid
14	Ataque Web: Injeção de SQL	SAT	SAT	inValid	inValid	inValid	inValid

Legenda: unSAT: Insatisfatível; SAT: Satisfatível; inValid: Consequência inválida; Valid: Consequência válida.

Tabela 5.4: Resultados dos testes de verificação formal para MLP-1

Capítulo 6

Conclusão

O objetivo principal deste trabalho foi aplicar a análise formal em redes neurais artificiais voltadas à cibersegurança. Para tanto, estabeleceu-se uma base teórica sólida abrangendo os conceitos de cibersegurança (redes digitais, ataques e sistemas de detecção de intrusão), redes neurais (MLP e aprendizado de máquina) e lógica (lógica proposicional, lógica de Łukasiewicz infinitamente-valorada e funções de McNaughton). Na sequência, foram detalhadas as técnicas de treinamento empregadas e os respectivos relatórios de desempenho estatístico, culminando na verificação das propriedades modeladas para as redes analisadas.

Os resultados obtidos na verificação formal evidenciaram que, para tarefas críticas como a detecção de intrusão, as redes neurais analisadas não garantiram propriedades de segurança consideradas relevantes. Dessa forma, demonstrou-se que a análise formal atua como uma ferramenta indispensável para a explicabilidade e confiabilidade desses modelos. Ficou comprovado que a dependência exclusiva de métricas estatísticas baseadas em amostras de teste é insuficiente para revelar falhas de comportamento que a análise lógica foi capaz de expor.

É importante ressaltar que os resultados apresentados estão condicionados às simplificações adotadas no escopo do projeto. A arquitetura dos MLPs foi limitada a cinco características de entrada e duas camadas ocultas com 16 neurônios cada. Especula-se que o treinamento de redes neurais mais profundas — com maior número de camadas e neurônios — ou com o uso de outras funções de custo possa resultar em modelos mais robustos e com maior compatibilidade com as propriedades de segurança modeladas, as quais deverão ser submetidas a novos processos de verificação.

Como sugestão para trabalhos futuros, recomenda-se a investigação de métodos para otimizar o desempenho computacional do processo de tradução da rede neural para o sistema lógico, visando escalabilidade. Além disso, sugere-se expandir o estudo para a representação e verificação formal de outras famílias de redes neurais além do perceptron de multicamadas. Sugere-se o desenvolvimento de novas fórmulas que representem propriedades essenciais da cibersegurança para serem submetidas à verificação. Além disso, devido à instabilidade numérica do solucionador **LUKASOL** para alguns testes, sugere-se a busca por implementações mais robustas de resolvedores para a lógica de Łukasiewicz.

Referências

- [C. BISHOP 2016] C.M. BISHOP. *Pattern Recognition and Machine Learning*. Information Science and Statistics. Springer New York, 2016. ISBN: 9781493938438. URL: <https://books.google.com.br/books?id=kOXDtAEACAAJ> (citado nas pgs. 11, 14).
- [M. BISHOP 2005] M. BISHOP. *Introduction to Computer Security*. Addison-Wesley, 2005. ISBN: 9780321247445. URL: <https://books.google.com.br/books?id=Z-IQAAAAMAAJ> (citado nas pgs. 3, 7).
- [BOFILL *et al.* 2019] Miquel BOFILL, Felip MANYÀ, Amanda VIDAL e Mateu VILLARET. “New complexity results for Łukasiewicz logic”. *Soft Computing* 23.7 (abr. de 2019). ISSN: 1433-7479, pp. 2187–2197. DOI: [10.1007/s00500-018-3365-9](https://doi.org/10.1007/s00500-018-3365-9). URL: <https://doi.org/10.1007/s00500-018-3365-9> (citado na pg. 27).
- [BURKOV 2019] A. BURKOV. *The Hundred-Page Machine Learning Book*. Andriy Burkov, 2019. ISBN: 9781999579517. URL: <https://books.google.com.br/books?id=0jbxwQEACAAJ> (citado nas pgs. 12, 13).
- [CAPUANO *et al.* 2022] Nicola CAPUANO, Giuseppe FENZA, Vincenzo LOIA e Claudio STANZIONE. “Explainable artificial intelligence in cybersecurity: a survey”. *IEEE Access* 10 (2022), pp. 93575–93600. DOI: [10.1109/ACCESS.2022.3204171](https://doi.org/10.1109/ACCESS.2022.3204171) (citado na pg. 17).
- [ENDERTON 2001] H.B. ENDERTON. *A Mathematical Introduction to Logic*. Academic Press, 2001. ISBN: 9780080496467. URL: https://books.google.com.br/books?id=dVncCI_EtUkC (citado na pg. 19).
- [FRIEDLANDER 1994] A. FRIEDLANDER. *Elementos de programação não-linear*. Coleção manuais. Editora da UNICAMP, 1994. ISBN: 9788526803046. URL: <https://books.google.com.br/books?id=kMCpAAAACAAJ> (citado na pg. 13).
- [GOODFELLOW *et al.* 2016] I. GOODFELLOW, Y. BENGIO e A. COURVILLE. *Deep Learning*. Adaptive Computation and Machine Learning series. MIT Press, 2016. ISBN: 9780262035613. URL: <https://books.google.com.br/books?id=Np9SDQAAQBAJ> (citado na pg. 16).

- [HÄHNLE 1994] Reiner HÄHNLE. “Many-valued logic and mixed integer programming”. *Annals of Mathematics and Artificial Intelligence* 12.3 (dez. de 1994). ISSN: 1573-7470, pp. 231–263. DOI: [10.1007/BF01530787](https://doi.org/10.1007/BF01530787) (citado na pg. 27).
- [HOOGLAND 2018] Donald HOOGLAND. *CICIDS2017*. <https://www.kaggle.com/datasets/dhoogla/cicids2017/data>. Acessado em: 06-12-2025. 2018 (citado na pg. 29).
- [KAUR *et al.* 2023] Ramanpreet KAUR, Dušan GABRIJELČIČ e Tomaž KLOBUČAR. “Artificial intelligence for cybersecurity: literature review and future research directions”. *Information Fusion* 97 (2023), p. 101804. ISSN: 1566-2535. DOI: <https://doi.org/10.1016/j.inffus.2023.101804>. URL: <https://www.sciencedirect.com/science/article/pii/S1566253523001136> (citado na pg. 4).
- [KINGMA e BA 2017] Diederik P. KINGMA e Jimmy BA. *Adam: A Method for Stochastic Optimization*. 2017. arXiv: [1412.6980](https://arxiv.org/abs/1412.6980) [cs.LG]. URL: <https://arxiv.org/abs/1412.6980> (citado na pg. 31).
- [KUROSE e ROSS 2017] J.F. KUROSE e K.W. ROSS. *Computer Networking: A Top-down Approach*. Pearson, 2017. ISBN: 9780133594140. URL: <https://books.google.com.br/books?id=OljpOAAACAAJ> (citado na pg. 4).
- [LIAO *et al.* 2013] Hung-Jen LIAO, Chun-Hung RICHARD LIN, Ying-Chih LIN e Kuang-Yuan TUNG. “Intrusion detection system: a comprehensive review”. *Journal of Network and Computer Applications* 36.1 (2013), pp. 16–24. ISSN: 1084-8045. DOI: [10.1016/j.jnca.2012.09.004](https://doi.org/10.1016/j.jnca.2012.09.004). URL: <https://www.sciencedirect.com/science/article/pii/S1084804512001944> (citado na pg. 7).
- [McNAUGHTON 1951] Robert McNAUGHTON. “A theorem about infinite-valued sentential logic”. *The Journal of Symbolic Logic* 16.1 (1951), pp. 1–13. ISSN: 00224812. URL: <http://www.jstor.org/stable/2268660> (acesso em 13/12/2025) (citado na pg. 17).
- [MORTARI 2001] C.A. MORTARI. *Introdução à lógica*. Ed. UNESP, 2001. ISBN: 9788571393370. URL: https://books.google.com.br/books?id=6iS4_QGZzHAC (citado na pg. 19).
- [PRETO 2022] Sandro PRETO. “Semantics modulo satisfiability with applications: function representation, probabilities and game theory”. *The Bulletin of Symbolic Logic* 28.2 (2022), pp. 264–265. DOI: [10.1017/bsl.2022.2](https://doi.org/10.1017/bsl.2022.2) (citado nas pgs. iii, v, 17, 23, 25, 35).
- [PRETO e FINGER 2020] Sandro PRETO e Marcelo FINGER. “An efficient algorithm for representing piecewise linear functions into logic”. *Electronic Notes in Theoretical Computer Science* 351 (2020). Proceedings of LSFA 2020, the 15th International Workshop on Logical and Semantic Frameworks, with Applications (LSFA 2020), pp. 167–186. ISSN: 1571-0661. DOI: <https://doi.org/10.1016/j.entcs.2020.08.009>. URL: <https://www.sciencedirect.com/science/article/pii/S1571066120300451> (citado na pg. 23).

- [PRETO e FINGER 2022a] Sandro PRETO e Marcelo FINGER. “Efficient representation of piecewise linear functions into łukasiewicz logic modulo satisfiability”. *Mathematical Structures in Computer Science* 32.9 (2022), pp. 1119–1144. DOI: [10.1017/S096012952200010X](https://doi.org/10.1017/S096012952200010X) (citado na pg. 23).
- [PRETO e FINGER 2022b] Sandro PRETO e Marcelo FINGER. “Proving properties of binary classification neural networks via łukasiewicz logic”. *Logic Journal of the IGPL* 31.5 (jun. de 2022), pp. 805–821. ISSN: 1367-0751. DOI: [10.1093/jigpal/jzac050](https://doi.org/10.1093/jigpal/jzac050). eprint: <https://academic.oup.com/jigpal/article-pdf/31/5/805/51754079/jzac050.pdf>. URL: <https://doi.org/10.1093/jigpal/jzac050> (citado na pg. 26).
- [PRETO e FINGER 2023] Sandro PRETO e Marcelo FINGER. “Effective reasoning over neural networks using Łukasiewicz logic”. In: *Compendium of Neurosymbolic Artificial Intelligence*. Vol. 369. Frontiers in Artificial Intelligence and Applications. Amsterdam: IOS Press, 2023. DOI: [10.3233/FAIA230160](https://doi.org/10.3233/FAIA230160). URL: <https://doi.org/10.3233/FAIA230160> (citado nas pgs. 25, 35).
- [PRETO e FINGER 2025] Sandro PRETO e Marcelo FINGER. *Regional, lattice and logical representations of neural networks*. 2025. DOI: [10.4204/EPTCS.421.4](https://doi.org/10.4204/EPTCS.421.4) (citado na pg. 25).
- [Preto e Finger] Sandro PRETO e Marcelo FINGER. “A polynomial-size logical twin for neural networks (a ser publicado)” () (citado nas pgs. 26, 35).
- [PRETO, MANYÀ *et al.* 2023] Sandro PRETO, Felip MANYÀ e Marcelo FINGER. “Benchmarking łukasiewicz logic solvers with properties of neural networks”. In: *2023 IEEE 53rd International Symposium on Multiple-Valued Logic (ISMVL)*. 2023, pp. 158–163. DOI: [10.1109/ISMVL57333.2023.00039](https://doi.org/10.1109/ISMVL57333.2023.00039) (citado na pg. 27).
- [ROSAY *et al.* 2020] Arnaud ROSAY, Florent CARLIER e Pascal LEROUX. “Mlp4nids: an efficient mlp-based network intrusion detection for cicids2017 dataset”. In: *Machine Learning for Networking*. Ed. por Selma BOUMERDASSI, Éric RENAULT e Paul MÜHLETHALER. Cham: Springer International Publishing, 2020, pp. 240–254. ISBN: 978-3-030-45778-5. DOI: [10.1007/978-3-030-45778-5_16](https://doi.org/10.1007/978-3-030-45778-5_16) (citado na pg. 17).
- [ROSENBLATT 1958] F ROSENBLATT. “The perceptron: a probabilistic model for information storage and organization in the brain”. en. *Psychological Review* 65.6 (1958), pp. 386–408. DOI: [10.1037/h0042519](https://doi.org/10.1037/h0042519) (citado na pg. 14).
- [SALAMA *et al.* 2023] Ramiz SALAMA, Fadi AL-TURJMAN, Manisha AERI e Satya Prakash YADAV. “Internet of intelligent things (iot) – an overview”. In: *2023 International Conference on Computational Intelligence, Communication Technology and Networking (CICTN)*. 2023, pp. 801–805. DOI: [10.1109/CICTN57981.2023.10141157](https://doi.org/10.1109/CICTN57981.2023.10141157) (citado na pg. 1).
- [SANTOS FAJARDO 2023] R.A. SANTOS FAJARDO. *Lógica Matemática*. EDUSP, 2023. ISBN: 9788531416729. URL: <https://books.google.com.br/books?id=Vjs1tAEACAAJ> (citado na pg. 20).

- [I. SHARAFALDIN *et al.* 2018] I. SHARAFALDIN, A. LASHKARI e A. GHORBANI. “Toward generating a new intrusion detection dataset and intrusion traffic characterization.” In: *Intl Conf. on Information Systems Security and Privacy (ICISSP)*. Vol. 1. Jan. de 2018, pp. 108–116. DOI: [10.5220/0006639801080116](https://doi.org/10.5220/0006639801080116) (citado na pg. 8).
- [Iman SHARAFALDIN *et al.* 2017] Iman SHARAFALDIN, Amirhossein GHARIB, Arash HABIBI LASHKARI e Ali GHORBANI. “Towards a reliable intrusion detection benchmark dataset”. *Software Networking 2017* (jan. de 2017), pp. 177–200. DOI: [10.13052/jsn2445-9739.2017.009](https://doi.org/10.13052/jsn2445-9739.2017.009) (citado na pg. 8).